Практическая работа №5. Машинное обучение в анализе данных с использованием Spark MLlib

1. Назначение практической работы

Цель работы — пройти полный цикл применения машинного обучения к табличным данным средствами Spark MLlib. Необходимо сформулировать задачу, подготовить признаки, разделить данные на обучающую, проверочную и тестовую части, обучить несколько моделей, оценить качество и объяснить полученный результат.

В предыдущих практиках уже рассматривались Spark DataFrame, ленивые вычисления, партиции, shuffle и Spark UI. Поэтому здесь не повторяется установка окружения. Основное внимание уделяется тому, как аналитик принимает решения: какие поля можно использовать, как не допустить утечку целевой информации, почему одна метрика не описывает модель полностью и как проверить, что сложный алгоритм действительно лучше простого правила.

Ожидаемые результаты обучения

- различать задачи классификации, регрессии и кластеризации по смыслу результата;
- выделять целевую переменную, числовые и категориальные признаки;
- строить цепочки предобработки категориальных данных для передачи их в модель;
- применять логарифмирование, масштабирование и полиномиальное расширение признаков;
- разделять данные на train, valid и test до обучения преобразований;
- обучать логистическую регрессию, случайный лес, линейную и полиномиальную регрессию, KMeans;
- вычислять метрики классификации и регрессии, строить графики ошибок и кривые качества;
- подбирать порог бинарного решения только по valid и выполнять итоговую оценку на test;
- оценивать кластеризацию по силуэту и содержательным профилям групп;
- сохранять обученный PipelineModel вместе с параметрами применения.

Исходные данные

В работе используются два набора данных UCI. Одна строка Online Shoppers описывает отдельную интернет-сессию. Одна строка Seoul Bike описывает один

час работы городского велопроката вместе с погодой и календарными признаками.

Таблица 1. Наборы данных практической работы

Задача	Файл	Размер	Целевой результат
Классификация	online_shoppers_intention.csv	12 330 × 18	Покупка / нет покупки
Регрессия	SeoulBikeData.csv	8 760 × 14	Число аренд за час
Кластеризация	online_shoppers_intention.csv	12 330 × 18	Группы похожих сессий

Online Shoppers содержит 10 422 сессии без покупки и 1 908 сессий с покупкой. Seoul Bike содержит 8 760 почасовых наблюдений; для регрессии далее остаются 8 465 часов, когда сервис реально работал.

Таблица 2. Алгоритмы основного сценария

Задача	Алгоритм	Реализация
Классификация	Логистическая регрессия	pyspark.ml.classification.LogisticRegression
Классификация	Случайный лес	pyspark.ml.classification.RandomForestClassifier
Регрессия	Линейная регрессия	pyspark.ml.regression.LinearRegression
Регрессия	Полиномиальная регрессия	PolynomialExpansion + LinearRegression
Регрессия	Случайный лес	pyspark.ml.regression.RandomForestRegressor
Кластеризация	KMeans	pyspark.ml.clustering.KMeans

> Контрольные таблицы и рисунки в этой методичке рассчитаны на приложенных CSV с теми же правилами разбиения и подготовки признаков. Во время подготовки документа для численной проверки использовалась независимая локальная реализация тех же моделей. При запуске кода Spark MLlib возможны небольшие отличия из-за другой реализации оптимизатора, инициализации деревьев и версии библиотек.

2. Основные понятия машинного обучения

2.1. Объект, признаки и целевая переменная

Объектом называют одну строку наблюдений. В этой практике объектом является либо час работы велопроката, либо отдельная интернет-сессия. Признаки — сведения, которыми модель машинного обучения может пользоваться для решения задачи; их объединяют в вектор x. Целевая переменная y — известный при обучении правильный результат: факт покупки или число аренд. Модель f с параметрами θ получает признаки и возвращает предсказание ŷ = fθ(x).

$$\hat{y} = f_\theta(\boldsymbol{x}).$$

В задаче классификации ответ относится к одному из классов. В задаче регрессии ответ является числом, и важен размер ошибки. В кластеризации правильные

классы для обучения не заданы: алгоритм ищет группы по сходству выбранных признаков. Наличие столбца Revenue в online_shoppers_intention.csv не превращает кластерную часть в классификацию, поскольку Revenue не поступает в алгоритм и подключается только после получения разбиения.

Параметры модели настраиваются алгоритмами по обучающим данным, для различных моделей используются свои оптимизаторы, в основном использующие особенности и механизмы градиентной оптимизации. Гиперпараметры, например, глубину дерева, число деревьев, силу регуляризации, число кластеров для моделей задаёт аналитик. Методика регуляризации ограничивает слишком сложное или неустойчивое решение, но она не исправляет неверно выбранные признаки и не устраняет утечку целевой информации.

В данной практике показывается возможное решение трех задач машинного обучения с использованием PySpark, а именно классификации, регрессии и кластеризации.

Таблица 3. Различие трёх задач

Задача	Что известно	Что возвращает алгоритм
Классификация	Метки классов в train	Класс и вероятность класса
Регрессия	Числовой ответ в train	Числовой прогноз
Кластеризация	Правильных классов нет	Технический номер группы

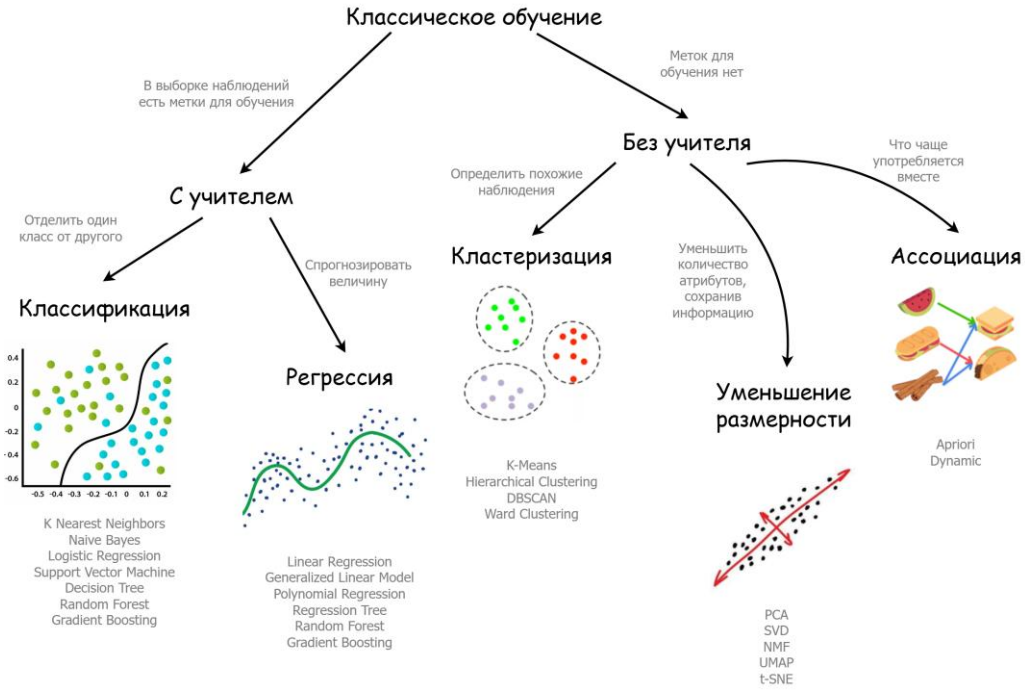


Рис. 1. Классические задачи машинного обучения

В задаче регрессии требуется предсказать непрерывное числовое значение. Для временных данных особенно важно соблюдать хронологический порядок: модель

обучается на прошлом и проверяется на более позднем периоде. Целевая переменная регрессии является количественной, поэтому качество оценивается величиной ошибки, а не долей совпавших классов.
В задаче классификации требуется предсказать класс — в этой работе бинарное значение 0/1 — и получить оценку уверенности модели в положительном классе. Целевая переменная дискретна, а окончательное решение зависит не только от score, но и от выбранного порога.
В задаче кластеризации каждой записи сопоставляется техническая метка группы близких по выбранным признакам объектов. Целевой метки для обучения нет, поэтому качество оценивают по геометрии разбиения и содержательной интерпретации профилей.

2.2. Train, valid и test

Обучающая часть выборки данных (train) используется для вычисления параметров модели и всех преобразований, которые зависят от распределения данных. Проверочная часть (valid) нужна для сравнения вариантов, выбора гиперпараметров и порога решения. Часть (test) остаётся закрытой до фиксации этих решений и используется для итоговой оценки. Повторное улучшение модели по (test) превращает его в ещё одну проверочную выборку.

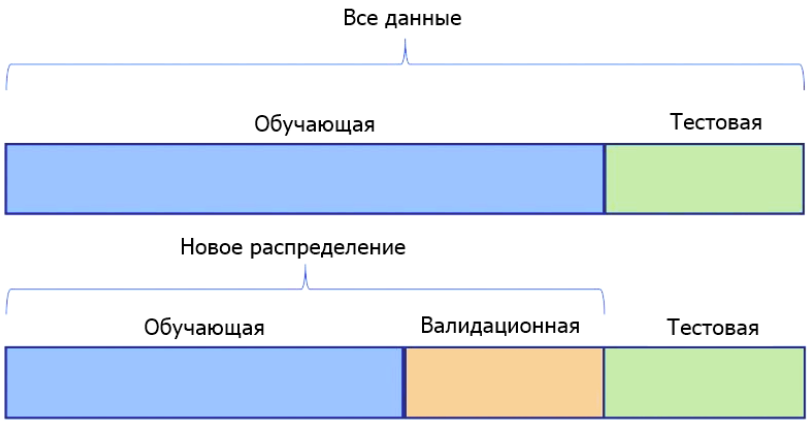


Рис. 2. Разбиение выборки на тренировочную, валидационную и тестовую

Порядок определения выборок является существенным из-за последовательности преобразований над данными. Сначала отделяют train, valid и test, затем обучают все преобразования на train. Среднее и стандартное отклонение, границы числовых интервалов и словарь категорий нельзя незаметно вычислять по всему набору. Одна и та же обученная цепочка применяется к valid и test без нового fit. При подборе гиперпараметров с перекрёстной проверкой (crossvalidation)

преобразования переобучаются внутри каждой итерации train разделенной выборки.

Разбиение должно соответствовать будущему использованию. Для независимых объектов допустимо случайное разбиение, а для будущих временных рядов нужно сохранять порядок времени.

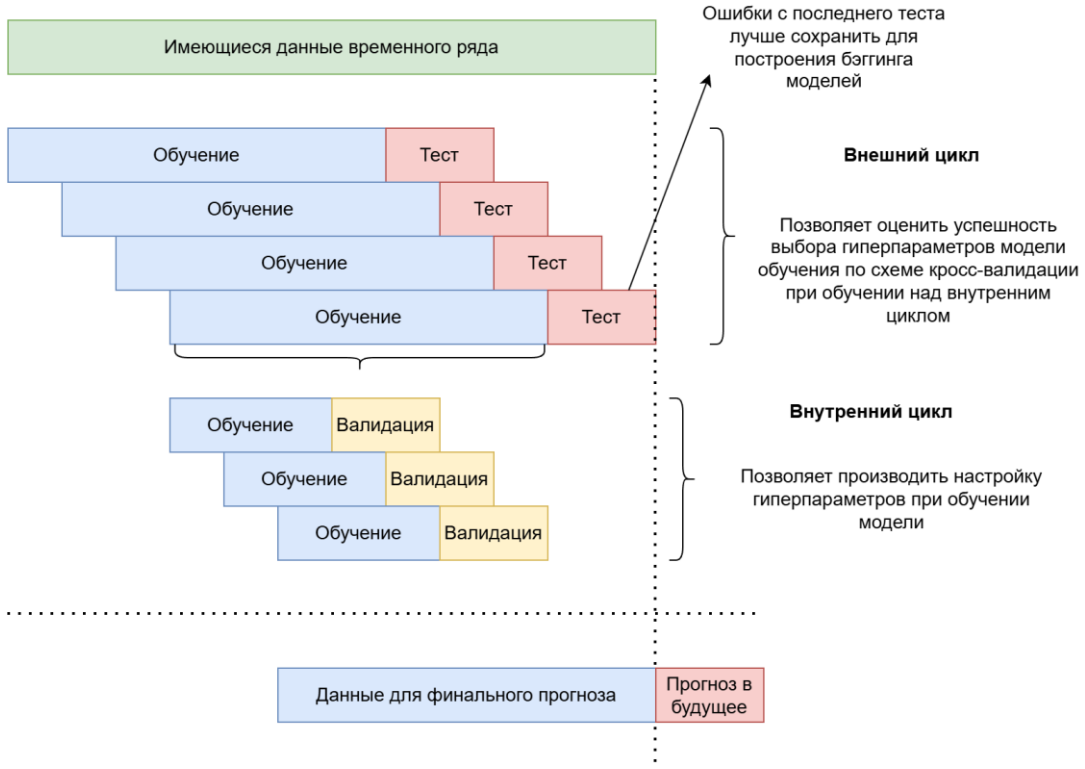


Рис. 3. Порядок обучения и тестирования обучения на временных рядах

Если один объект представлен несколькими строками, полезно отделять целые объекты, а не отдельные записи. Отсутствие идентификатора клиента — ограничение данных, а не основание считать записи независимыми.

> Словарь категорий, стандартное отклонение, границы интервалов и другие статистики также обучаются только на train. Поэтому их нужно помещать внутрь Pipeline, а не вычислять заранее по всей таблице.

2.3. Утечка данных, переобучение и простое правило сравнения

Утечка данных возникает, когда модель получает сведения, которых не будет в момент принятия решения, или когда информация из valid либо test влияет на обучение. В Online Shoppers доступность PageValues зависит от момента выдачи прогноза: для оценки по завершённой сессии поле допустимо, а для раннего прогноза может быть недоступно. В Seoul Bike фактическая погода наблюдаемого часа не равна заранее известному прогнозу погоды. Высокая метрика с такими

признаками может описывать удобную ретроспективную постановку, но не гарантировать пригодность решения в эксплуатации.

Переобучение означает, что модель хорошо подстраивается под обучающие наблюдения, но заметно хуже работает на новых. Сравнение ошибки train и valid помогает это заметить, однако разрыв может быть связан и с изменением данных во времени. Недообучение проявляется, когда выбранное представление или модель не улавливают существенные зависимости. Поэтому ошибка сама по себе ещё не объясняет причину.

До сложного алгоритма необходимо построить простую модель не на основе обучения, а с помощью базовых или когортных статистик. Например, всегда выбирать частый класс в классификации или в регрессии предсказывать целевое значение средним целевым значением на train или средними для похожих категорий (например предсказание цены поездки в 18:00 средним ценником по истории поездок в 18:00). Такое правило называют ***базовым решением***. Если сложная модель не лучше него по нужному показателю, увеличение вычислительных затрат пока не обосновано.

2.4. Предобработка данных

Номер категории не является её величиной. Код браузера 8 не означает «вдвое больше браузера 4». StringIndexer назначает категориям числовые индексы порядкового номера, а OneHotEncoder переводит их в набор индикаторов векторных присутствия категории:

$$browser\text{: [firefox, chrome, firefox, brave]} \rightarrow^{SI} [0, 1,0,2]$$

$$browser\text{: [firefox, chrome, firefox, brave]} \rightarrow^{OHE} \begin{pmatrix} [1, 0,0] \\ [0,1,0] \\ [1,0,0] \\ [0,01] \end{pmatrix}$$

StandardScaler приводит признаки к сопоставимому масштабу. Это особенно важно для линейных моделей с регуляризацией и для методов, основанных на расстоянии. Масштабирование делит числовые признаки на характерный разброс:

$$\tilde{\boldsymbol{x}} = \frac{\boldsymbol{x} - mean(\boldsymbol{x})}{std(\boldsymbol{x})}.$$

Это важно для линейных моделей с регуляризацией и алгоритмов, использующих расстояния. Для сильно скошенных счётчиков и длительностей применяется $\log1p(x) = \ln(1 + x)$, тогда нулевые значения сохраняются, а длинный правый хвост становится короче.

> Pipeline хранит правильный порядок преобразований и гарантирует, что valid, test и новые данные проходят через те же словари и масштабы, которые были получены на train.

2.5. Метрики

Для классификации используются *precision, recall, F1, ROC-AUC* и *PR-AUC*. Метрика *accuracy* при несбалансированном классе может быть обманчивой: правило «всегда нет покупки» уже даёт больше правильных ответов, но не находит ни одной покупки.

Для регрессии MAE показывает средний модуль ошибки в прогнозах. RMSE сильнее наказывает крупные промахи. R^2 сравнивает модель с постоянным прогнозом, равным среднему фактическому значению оцениваемой части. На новых данных R^2 может быть отрицательным.

Для кластеризации используется silhouette. Он сравнивает близость объекта к своему кластеру и к ближайшему соседнему кластеру. Большое значение помогает выбрать k, но не заменяет проверку размеров и смысла групп.

2.6. Estimator, Transformer и Pipeline в Spark MLlib

В современной части MLlib используется DataFrame-интерфейс pyspark.ml. Объект Estimator обучается по данным методом fit и возвращает обученную модель. Transformer не подбирает параметры при каждом применении, а преобразует DataFrame методом transform. Например, LogisticRegression и StandardScaler являются Estimator до обучения, а полученные LogisticRegressionModel и StandardScalerModel — Transformer.

Объект Pipeline задаёт последовательность подготовки и обучения:

1) логарифмирование и создание признаков,
2) индексация категорий,
3) one-hot-кодирование,
4) сборка вектора,
5) масштабирование при необходимости,
6) и, наконец, модель.

После fit получается PipelineModel, в котором уже сохранены словари категорий, статистики масштабирования и параметры алгоритма. Поэтому valid, test и новые строки проходят через ту же обученную цепочку, а не через заново рассчитанную предобработку.

В типовом Spark ML DataFrame столбец features содержит вектор признаков, label — правильный ответ, prediction — стандартное решение модели. Классификатор дополнительно возвращает probability и rawPrediction. В этой работе вероятность класса 1 извлекается в столбец score, а собственный порог применяется отдельно

и записывается как решение. Это позволяет менять прикладной порог без переобучения модели.

Вызов fit запускает реальные распределённые вычисления, в котором индексаторы собирают словари, scaler вычисляет статистики, а алгоритм выполняет проходы по данным для получения ошибок и настройки. Один Pipeline.fit может породить несколько Spark Jobs. Преобразования transform обычно строят ленивый план, а вычисление начинается при вызове расчета метрики, show(), записи или другом action.

Spark DataFrame остаётся распределённым, пока данные не собраны на драйвер. collect() и toPandas() допустимы только для заранее ограниченных таблиц, среди которых метрики, строки для графика или небольшие выборки. Основные наборы и обучение моделей в этой работе остаются в Spark.

3. Чтение и контроль исходных данных

Сессия Spark считается уже созданной. Первый блок подключает только библиотеки, задаёт пути и проверяет, что оба CSV доступны. Локальные библиотеки pandas, NumPy, Matplotlib и scikit-learn используются лишь для небольших итоговых таблиц и графиков. Обучение моделей выполняется через Spark MLlib.

Практический блок 3.1. Импорты и рабочие каталоги

```
from pathlib import Path
from time import perf_counter
import csv
import json
import math

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from pyspark.sql import functions as F, Window
from pyspark.sql.types import (
    StructType, StructField, LongType, IntegerType,
    DoubleType, StringType, BooleanType
)
from pyspark.ml import Pipeline, PipelineModel
from pyspark.ml.functions import vector_to_array
from pyspark.ml.feature import (
    SQLTransformer, StringIndexer, OneHotEncoder,
    VectorAssembler, StandardScaler, PolynomialExpansion, PCA
)
from pyspark.ml.classification import LogisticRegression,
RandomForestClassifier
from pyspark.ml.regression import LinearRegression, RandomForestRegressor
from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import BinaryClassificationEvaluator,
RegressionEvaluator
```

```
from sklearn.metrics import (
    precision_recall_curve, roc_curve, confusion_matrix,
    silhouette_score
)

SEED = 42
ROOT = Path.cwd()
WORK = ROOT / "practice6_data"
OUT = ROOT / "practice6_results"
WORK.mkdir(exist_ok=True)
OUT.mkdir(exist_ok=True)

SHOP_SOURCE = ROOT / "online_shoppers_intention.csv"
BIKE_SOURCE = ROOT / "SeoulBikeData.csv"

for path in [SHOP_SOURCE, BIKE_SOURCE]:
    if not path.is_file():
        raise FileNotFoundError(f"Не найден файл: {path}")

sc = spark.sparkContext
sc.setLogLevel("WARN")
print("Spark:", spark.version)
print("Master:", sc.master)
print("Spark UI:", sc.uiWebUrl)
```

Переменная sc указывает на уже работающий SparkContext. Адрес Spark UI выводится фактически, а не задаётся вручную: порт может отличаться от 4040, если другая сессия уже использовала его.

Практический блок 3.2. Устойчивый row_id для интернет-сессий

```
SHOP_WORK = WORK / "online_shoppers_with_id.csv"

with SHOP_SOURCE.open("r", encoding="utf-8-sig", newline="") as src, \
     SHOP_WORK.open("w", encoding="utf-8", newline="") as dst:
    reader = csv.reader(src)
    writer = csv.writer(dst)
    header = next(reader)
    writer.writerow(["row_id"] + header)
    for row_id, row in enumerate(reader, start=1):
        writer.writerow([row_id] + row)
```

Колонка row_id повторяет исходный порядок CSV и не входит в features. Она нужна для воспроизводимого разбиения, сохранения предсказаний и соединения кластерных меток с исходной таблицей.

Практический блок 3.3. Явные схемы и чтение CSV

```
shop_schema = StructType([
    StructField("row_id", LongType(), False),
    StructField("Administrative", DoubleType(), False),
    StructField("Administrative_Duration", DoubleType(), False),
    StructField("Informational", DoubleType(), False),
    StructField("Informational_Duration", DoubleType(), False),
    StructField("ProductRelated", DoubleType(), False),
    StructField("ProductRelated_Duration", DoubleType(), False),
```

```
    StructField("BounceRates", DoubleType(), False),
    StructField("ExitRates", DoubleType(), False),
    StructField("PageValues", DoubleType(), False),
    StructField("SpecialDay", DoubleType(), False),
    StructField("Month", StringType(), False),
    StructField("OperatingSystems", IntegerType(), False),
    StructField("Browser", IntegerType(), False),
    StructField("Region", IntegerType(), False),
    StructField("TrafficType", IntegerType(), False),
    StructField("VisitorType", StringType(), False),
    StructField("Weekend", BooleanType(), False),
    StructField("Revenue", BooleanType(), False),
])

bike_schema = StructType([
    StructField("date_text", StringType(), False),
    StructField("label", DoubleType(), False),
    StructField("hour", IntegerType(), False),
    StructField("temperature", DoubleType(), False),
    StructField("humidity", DoubleType(), False),
    StructField("wind_speed", DoubleType(), False),
    StructField("visibility", DoubleType(), False),
    StructField("dew_point", DoubleType(), False),
    StructField("solar_radiation", DoubleType(), False),
    StructField("rainfall", DoubleType(), False),
    StructField("snowfall", DoubleType(), False),
    StructField("season", StringType(), False),
    StructField("holiday", StringType(), False),
    StructField("functioning_day", StringType(), False),
])

shop_raw = (spark.read
    .option("header", True)
    .option("mode", "FAILFAST")
    .schema(shop_schema)
    .csv(SHOP_WORK.resolve().as_posix()))

bike_raw = (spark.read
    .option("header", True)
    .option("encoding", "ISO-8859-1")
    .option("enforceSchema", True)
    .option("mode", "FAILFAST")
    .schema(bike_schema)
    .csv(BIKE_SOURCE.resolve().as_posix()))

print("Online Shoppers:", shop_raw.count(), "строк")
print("Seoul Bike:", bike_raw.count(), "строк")
shop_raw.printSchema()
bike_raw.printSchema()
```

Файл данных SeoulBikeData.csv использует однобайтовую кодировку для символа градуса в заголовке, поэтому чтение выполняется с encoding="ISO-8859-1". Безопасные имена схемы назначаются по порядку столбцов. При замене файла порядок необходимо проверить заново.

Практический блок 3.4. Контроль строк, ключей и пропусков

```
def null_counts(df):
    return df.select(*[
        F.sum(F.col(c).isNull().cast("int")).alias(c)
        for c in df.columns
    ]).first().asDict()
```

```
shop_raw.groupBy("Revenue").count().orderBy("Revenue").show()
bike_raw.groupBy("functioning_day").agg(
    F.count("*").alias("n"),
    F.min("label").alias("min_target"),
    F.max("label").alias("max_target")
).show()
```

> 295 нерабочих часов нельзя автоматически считать обычными наблюдениями низкого спроса: ноль появился из-за закрытия сервиса. Основная регрессия ниже отвечает на другой вопрос — сколько велосипедов арендуют, когда сервис работает.

4. Классификация покупательского намерения

4.1. Постановка и дисбаланс классов

Цель классификации — оценить вероятность ***Revenue = True***. Положительный класс отмечаем редким, в выборке покупкой заканчивается около 15,5% сессий. Поэтому модели сравниваются прежде всего по PR-AUC, precision, recall и F1, а не только по accuracy.

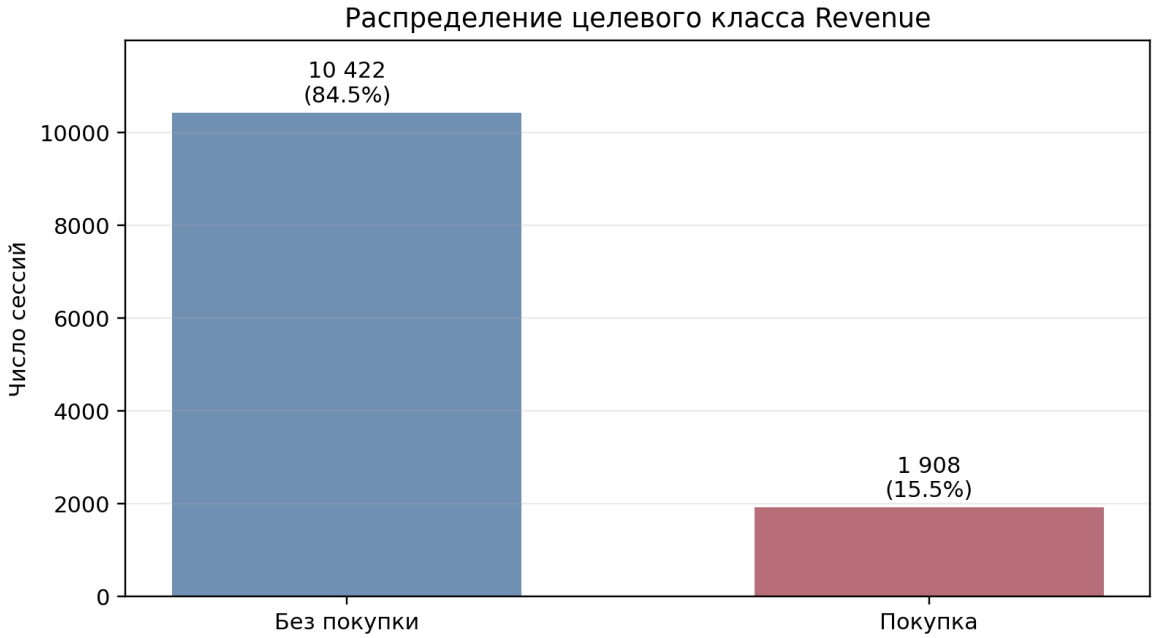


Рис. 4. Распределение классов Online Shoppers

Диаграмма показывает исходный дисбаланс: столбец «Без покупки» во много раз выше столбца «Покупка». Это ещё не результат модели, а характеристика задачи. Постоянный ответ «нет покупки» дал бы accuracy около 84,5 %, но recall положительного класса был бы равен нулю. Поэтому далее оценивается не только доля правильных ответов, но и способность находить редкие покупки.

4.2. Момент доступности признаков

Признак PageValues сильно связан с покупкой. В основном опыте он включён: сценарий предполагает оценку по накопленному состоянию сессии. Но при раннем прогнозе это поле может быть ещё не рассчитано в той же форме. Поэтому после основного опыта выполняется абляция — повторное обучение без PageValues.

PageValues не объявляется автоматически «запрещённым» признаком. Его допустимость определяется сценарием. В основном опыте прогноз строится по накопленному состоянию сессии, а отдельный опыт без PageValues показывает, насколько вывод зависит от этого поля. Такое исключение одного фактора называют абляцией; оно проверяет чувствительность решения, но само по себе не доказывает причинную связь.

> Высокая важность признака не доказывает причинное влияние. Она означает только, что модель часто использует этот столбец для разделения наблюдений в предоставленном наборе.

4.3. Преобразование, разбиение и веса классов

Шесть счётчиков и длительностей логарифмируются. OperatingSystems, Browser, Region и TrafficType рассматриваются как категории, хотя в CSV они имеют целочисленный тип. Разбиение выполняется отдельно внутри каждого класса по SHA-256 от row_id и seed. Это даёт одинаковый состав частей при повторном запуске.

Списки SHOP_LOG_SOURCE, SHOP_NUM_BASE и SHOP_CAT явно фиксируют состав будущего вектора. Шесть счётчиков и длительностей имеют длинный правый хвост, поэтому внутри Pipeline для них создаётся log1p-преобразование. Нулевые значения при этом остаются нулевыми, а очень крупные наблюдения меньше доминируют над остальными.

Числовые коды OperatingSystems, Browser, Region и TrafficType приводятся к строкам, потому что это названия категорий, а не величины. Затем внутри каждого класса строки упорядочиваются по воспроизводимому хешу row_id и seed. Такое стратифицированное разбиение сохраняет редкий положительный класс во всех частях и не зависит от текущего порядка партиций.

После разбиения веса вычисляются только по shop_train. Для класса с используется правило $w_c = N / (K \cdot N_c)$, где N — размер train, K = 2 — число классов, N_c — число строк данного класса. Редкий класс получает больший вес, поэтому ошибка на покупке сильнее влияет на функцию потерь. Фактическое число строк при этом не меняется.

```
SHOP_LOG_SOURCE = [
    "Administrative", "Administrative_Duration",
    "Informational", "Informational_Duration",
    "ProductRelated", "ProductRelated_Duration"
]
SHOP_LOG = ["log_" + c for c in SHOP_LOG_SOURCE]
SHOP_NUM_BASE = SHOP_LOG + ["BounceRates", "ExitRates", "SpecialDay"]
SHOP_CAT = [
    "Month", "OperatingSystems", "Browser", "Region",
    "TrafficType", "VisitorType", "Weekend"
]

shop = shop_raw.withColumn("label", F.col("Revenue").cast("double"))
for c in ["OperatingSystems", "Browser", "Region", "TrafficType", "Weekend"]:
    shop = shop.withColumn(c, F.col(c).cast("string"))

order_key = F.sha2(F.concat_ws(
    ":", F.col("row_id").cast("string"), F.lit(str(SEED))
), 256)
rank_window = Window.partitionBy("label").orderBy(order_key, "row_id")
class_window = Window.partitionBy("label")

shop_split = (shop
    .withColumn("_rank", F.row_number().over(rank_window))
    .withColumn("_size", F.count("*").over(class_window))
    .withColumn(
        "split",
        F.when(F.col("_rank") <= F.floor(F.col("_size") * 0.60), "train")
         .when(F.col("_rank") <= F.floor(F.col("_size") * 0.80), "valid")
         .otherwise("test")
    )
    .drop("_rank", "_size")
    .cache())

shop_split.groupBy("split", "label").count().orderBy("split", "label").show()

shop_train = shop_split.filter("split = 'train'").drop("split")
shop_valid = shop_split.filter("split = 'valid'").drop("split")
shop_test = shop_split.filter("split = 'test'").drop("split")

counts = {int(r["label"]): r["count"]
          for r in shop_train.groupBy("label").count().collect()}
train_n = sum(counts.values())
weight_0 = train_n / (2.0 * counts[0])
weight_1 = train_n / (2.0 * counts[1])
shop_train = shop_train.withColumn(
    "class_weight",
    F.when(F.col("label") == 1.0, F.lit(weight_1)).otherwise(F.lit(weight_0))
)
print("Веса классов:", weight_0, weight_1)
```

Таблица 4. Состав частей классификации

Часть	Нет покупки	Покупка	Всего
Train	6 253	1 144	7 397
Valid	2 084	382	2 466
Test	2 085	382	2 467

Таблица подтверждает, что во всех трёх частях присутствуют оба класса и сохраняется близкая доля покупок. Далее обе модели обучаются и сравниваются на одном и том же составе train и valid. Новое разбиение внутри цикла по алгоритмам создало бы некорректное сравнение, поскольку изменились бы одновременно и модель, и проверочные объекты.
В train положительный класс получает больший вес. Вес не меняет фактическое число строк, но усиливает штраф за ошибки на редком классе. Это помогает модели не свести решение к постоянному отрицательному ответу.

4.4. Логистическая регрессия и случайный лес

Несмотря на название, ***логистическая регрессия*** в этой задаче решает классификацию. Она вычисляет линейную комбинацию признаков и преобразует её в число от 0 до 1. Граница классов зависит от порога, одна и та же вероятность может привести к разному решению при порогах 0,5 и 0,2.

$$p(y = 1 \mid x) = 1 / [1 + exp(-z)], \quad z = b + \Sigma_j w_j x_j$$

Коэффициенты обучаются так, чтобы согласовать вероятности с наблюдаемыми классами. В работе применяется L2-регуляризация, в процессе обучения большие коэффициенты получают дополнительный штраф к функции стоимости модели. Числовые признаки масштабируются, чтобы единицы измерения меньше влияли на действие штрафа. Стандартное отклонение для масштабирования оценивается только на train выборке, а центрирование отключено, чтобы не превращать разреженный вектор категорий в плотный.
Логистическая регрессия линейна относительно подготовленного вектора features. Это не означает линейность по исходной таблице: one-hot-кодирование и log1p уже изменили представление данных. Коэффициент показывает изменение логарифма отношения шансов при фиксированных остальных координатах. Для взаимосвязанных признаков и категориальных индикаторов отдельный знак нельзя трактовать как самостоятельный причинный эффект.
Дерево решений последовательно проверяет условия по признакам, например, попадает ли значение в определённый диапазон. Оно строит ветви с помощью тренировочной выборке, сохраняя уверенности в классе в каждом листе на основе соотношений в них попавшей выборки. Разбиения строятся от признака, допускающего лучшее ступенчатое отделение классов, таким образом, чтобы максимизировать *разницу энтропии* или *коэффициента Джини* до и после разделения. Все принятия решений о построении нового разделения настраиваются гиперпараметрами перед обучением fit().

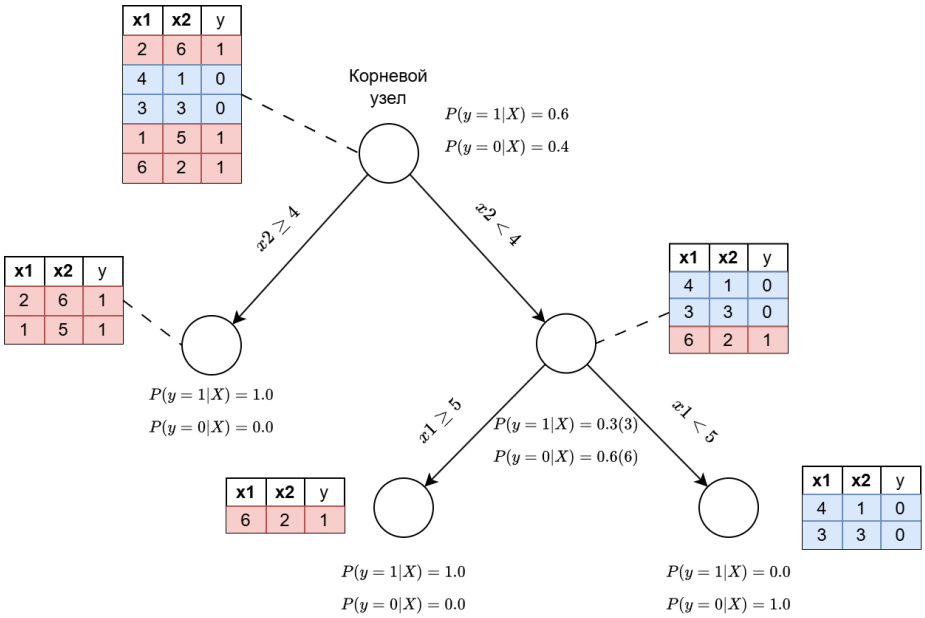


Рис. 5. Принцип работы дерева решений

Случайный лес объединяет несколько деревьев, построенных со случайностью в выборе данных и признаков. Для классификации итоговая вероятность получается из предсказаний деревьев. Такой подход позволяет учитывать нелинейные сочетания признаков без ручного перечисления всех взаимодействий.

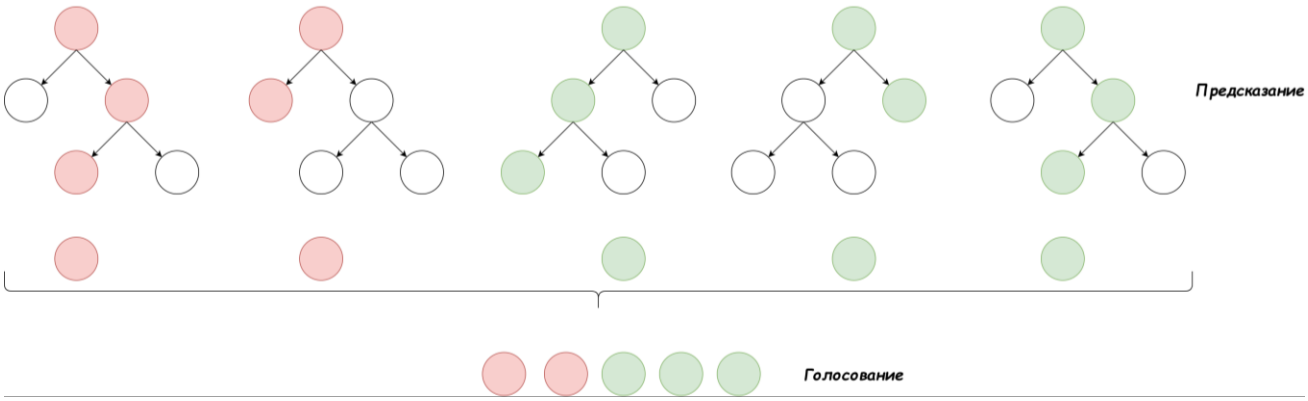


Рис. 6. Предсказание случайным лесом

При обучении каждое дерево получает отличающийся случайный набор наблюдений и рассматривает случайное подмножество признаков при поиске разбиения. Такая вариативность уменьшает зависимость ансамбля от одного конкретного дерева. Итоговый score положительного класса формируется усреднением ответов деревьев. Он пригоден для ранжирования и выбора порога, но не гарантирует идеальную вероятностную калибровку.

Глубина ограничивает сложность каждого дерева, а число деревьев — размер ансамбля и стоимость обучения. В коде используются 80 деревьев глубины не более 8, до 64 вариантов разбиения категориальных координат и случайное

подмножество признаков sqrt в каждом узле. Масштабирование числовых предикторов для обучения случайного леса не выполняется, так как при построении деревья не настраивают множители, а сравнивают значения с порогами и не требуют одинаковых единиц измерения. Однако ошибки кодирования категорий, утечка и недостаток примеров остаются важными и для модели случайного леса.

Практический блок 4.5. Pipeline двух классификаторов

Функция shop_pipeline собирает полное решение, а не только последний классификатор. SQLTransformer создаёт логарифмированные поля; StringIndexer обучает словари категорий на train; OneHotEncoder превращает индексы в индикаторные векторы; VectorAssembler объединяет числовые и категориальные координаты.

В ветке LR собранный вектор дополнительно масштабируется и передаётся в LogisticRegression с L2-регуляризацией и весами классов. В ветке RF масштабирование пропускается, а RandomForestClassifier получает тот же допустимый набор исходных полей в представлении, подходящем для деревьев. Параметр include_page_values позволяет повторить тот же эксперимент без PageValues, не переписывая остальные этапы.

Вспомогательная функция binary_scores извлекает probability[1] как score класса «Покупка». Функция binary_metrics не переносит все предсказания в Python, так как Spark агрегирует TP, FP, FN и TN, после чего в драйвер возвращаются четыре числа и рассчитанные по ним показатели. Площади под ROC- и PR-кривыми вычисляются тем же набором оценок.

```
def shop_pipeline(kind, include_page_values=True):
    expressions = [
        f"log1p(`{source}`) AS `{target}`"
        for source, target in zip(SHOP_LOG_SOURCE, SHOP_LOG)
    ]
    log_stage = SQLTransformer(
        statement="SELECT *, " + ", ".join(expressions) + " FROM __THIS__"
    )

    index_cols = [c + "_idx" for c in SHOP_CAT]
    encoded_cols = [c + "_ohe" for c in SHOP_CAT]
    indexer = StringIndexer(
        inputCols=SHOP_CAT,
        outputCols=index_cols,
        handleInvalid="keep",
        stringOrderType="alphabetAsc"
    )
    encoder = OneHotEncoder(
        inputCols=index_cols,
        outputCols=encoded_cols,
        handleInvalid="keep",
        dropLast=False
    )
```

```
    numeric = SHOP_NUM_BASE + (["PageValues"] if include_page_values else [])
    if kind == "LR":
        assembler = VectorAssembler(
            inputCols=numeric + encoded_cols,
            outputCol="features_raw"
        )
        scaler = StandardScaler(
            inputCol="features_raw", outputCol="features",
            withMean=False, withStd=True
        )
        estimator = LogisticRegression(
            maxIter=100, regParam=0.05, elasticNetParam=0.0,
            standardization=False, weightCol="class_weight"
        )
        stages = [log_stage, indexer, encoder, assembler, scaler, estimator]
    elif kind == "RF":
        assembler = VectorAssembler(
            inputCols=numeric + encoded_cols,
            outputCol="features"
        )
        estimator = RandomForestClassifier(
            numTrees=80, maxDepth=8, maxBins=64,
            featureSubsetStrategy="sqrt", seed=SEED,
            weightCol="class_weight"
        )
        stages = [log_stage, indexer, encoder, assembler, estimator]
    else:
        raise ValueError(kind)
    return Pipeline(stages=stages)


def binary_scores(model, df):
    return (model.transform(df)
        .withColumn("score", vector_to_array("probability")[1])
        .select("row_id", "label", "score"))


def binary_metrics(pred, threshold=0.5, ranking=True):
    decision = F.col("score") >= F.lit(float(threshold))
    y = F.col("label") == 1.0
    values = pred.agg(
        F.sum((decision & y).cast("long")).alias("tp"),
        F.sum((decision & ~y).cast("long")).alias("fp"),
        F.sum((~decision & y).cast("long")).alias("fn"),
        F.sum((~decision & ~y).cast("long")).alias("tn")
    ).first().asDict()

    tp, fp, fn, tn = (values[k] for k in ["tp", "fp", "fn", "tn"])
    n = tp + fp + fn + tn
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall
else 0.0
    result = {
        "accuracy": (tp + tn) / n,
        "precision": precision,
        "recall": recall,
        "f1": f1,
        "tp": tp, "fp": fp, "fn": fn, "tn": tn
    }
    if ranking:
        for metric in ["areaUnderROC", "areaUnderPR"]:
            result[metric] = BinaryClassificationEvaluator(
                labelCol="label", rawPredictionCol="score", metricName=metric
```

```
            ).evaluate(pred)
    return result
```

Создание Pipeline ещё не обучает ни индексаторы, ни scaler, ни модель. Все параметры определяются только при fit(shop_train). Затем один и тот же PipelineModel применяется к valid и test через transform. Благодаря этому словари категорий и масштаб числовых координат не получают информацию из проверочных данных.

Функция создаёт новый Pipeline при каждом вызове. Это важно: один и тот же объект Estimator не должен незаметно хранить параметры предыдущего опыта. В LR scaler обучается на train и входит в сохраняемую цепочку; для RF масштабирование не требуется.

4.6. Метрики и обучение

Истинно положительный ответ TP — модель выбрала класс «Покупка», и сессия действительно завершилась покупкой. FP — модель ожидала покупку, но её не было. FN — покупка произошла, однако модель её пропустила. TN — верно предсказанное отсутствие покупки. Precision отвечает на вопрос «какая доля положительных решений подтвердилась?», recall — «какую долю всех покупок удалось найти?».

$$Accuracy = (TP + TN) / n, \quad Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN), \quad F1 = 2 \cdot TP / (2TP + FP + FN)$$

В работе F1 считается именно для положительного класса. Усреднённый по частотам классов показатель может скрыть слабое обнаружение покупок. Если знаменатель precision, recall или F1 равен нулю, код возвращает 0. Сам факт отсутствия положительных решений всё равно нужно обсудить.

ROC-AUC и площадь под кривой precision–recall оценивают упорядочивание объектов по score при разных порогах отделения 0 от 1. Для редких покупок основной показатель выбора модели здесь — areaUnderPR. Его значение зависит от распространённости положительного класса, поэтому сравнение проводится на одной и той же valid. Метрику areaUnderPR в Spark не следует автоматически отождествлять с любой реализацией average precision в другой библиотеке. Площадь под кривой и качество вероятностной калибровки также не одно и то же.

Цикл обучает обе полные цепочки на одном shop_train. Поле setJobGroup задаёт понятные подписи в Spark UI, объект perf_counter измеряет время fit всей цепочки, а предсказания valid кэшируются, поскольку используются далее для таблицы, выбора порога и построения кривых. Победитель выбирается по areaUnderPR на valid, а не по test.

```
shop_models = {}
shop_valid_predictions = {}
shop_rows = []

for name, kind in [("Логистическая регрессия", "LR"),
                   ("Случайный лес", "RF")]:
    sc.setJobGroup("P6_CLASS_" + kind, "Классификация: " + name)
    t0 = perf_counter()
    model = shop_pipeline(kind, include_page_values=True).fit(shop_train)
    fit_s = perf_counter() - t0
    pred = binary_scores(model, shop_valid).cache()
    pred.count()
    row = {"model": name, "fit_s": fit_s, **binary_metrics(pred)}
    shop_models[name] = model
    shop_valid_predictions[name] = pred
    shop_rows.append(row)

shop_validation = pd.DataFrame(shop_rows).sort_values("areaUnderPR",
ascending=False)
print(shop_validation.round(4).to_string(index=False))
BEST_SHOP_NAME = str(shop_validation.iloc[0]["model"])
BEST_SHOP = shop_models[BEST_SHOP_NAME]
```

Таблица 5. Контрольный результат на valid при пороге 0,5

Модель	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Логистическая регрессия	0.842	0.494	0.780	0.605	0.907	0.634
Случайный лес	0.873	0.561	0.840	0.673	0.927	0.711

Контрольная таблица показывает одновременно ранжирующие метрики и показатели при пороге 0,5. Случайный лес имеет более высокие PR-AUC, ROC-AUC и F1, но ни одна строка не должна читаться изолированно: высокий recall сопровождается заметным числом FP. В прикладной задаче этот компромисс связывают со стоимостью лишнего показа предложения и ценой пропущенной покупки.

Случайный лес показывает более высокие PR-AUC и F1. Обе модели находят большую часть покупок, но платят за это ложными положительными решениями. В прикладной задаче такой компромисс нужно связать со стоимостью показа предложения или работы оператора.

4.7. Выбор порога и однократный test

Порог выбирается только на valid. После его фиксации test используется один раз. Для выбранной модели таблица valid переносится в pandas: здесь всего 2 466 строк и два числовых столбца, поэтому операция безопасна для памяти драйвера. precision_recall_curve перебирает все возможные границы score, после чего для

каждой точки вычисляется F1. Порог фиксируется в максимуме F1 на valid и больше не изменяется по результатам test.

```
valid_pd = shop_valid_predictions[BEST_SHOP_NAME].toPandas()
precision, recall, thresholds = precision_recall_curve(
    valid_pd["label"], valid_pd["score"]
)
f1_values = 2 * precision * recall / np.maximum(precision + recall, 1e-12)
best_index = int(np.nanargmax(f1_values[:-1]))
SHOP_THRESHOLD = float(thresholds[best_index])
print("Выбранная модель:", BEST_SHOP_NAME)
print("Порог по максимуму F1 на valid:", round(SHOP_THRESHOLD, 4))
print(binary_metrics(
    shop_valid_predictions[BEST_SHOP_NAME], SHOP_THRESHOLD, ranking=True
))

shop_test_scores = binary_scores(BEST_SHOP, shop_test).cache()
shop_test_scores.count()
print("TEST:", binary_metrics(shop_test_scores, SHOP_THRESHOLD, ranking=True))
```

```
Выбранная модель: Случайный лес
Порог по максимуму F1 на valid: 0,550
TEST: accuracy=0,876; precision=0,577; recall=0,759; F1=0,655; ROC-AUC=0,905;
PR-AUC=0,664
Матрица ошибок: TN=1872, FP=213, FN=92, TP=290
```

В итоговом тесте модель находит 290 из 382 покупок и пропускает 92. Одновременно 213 сессий без покупки получают положительное решение.

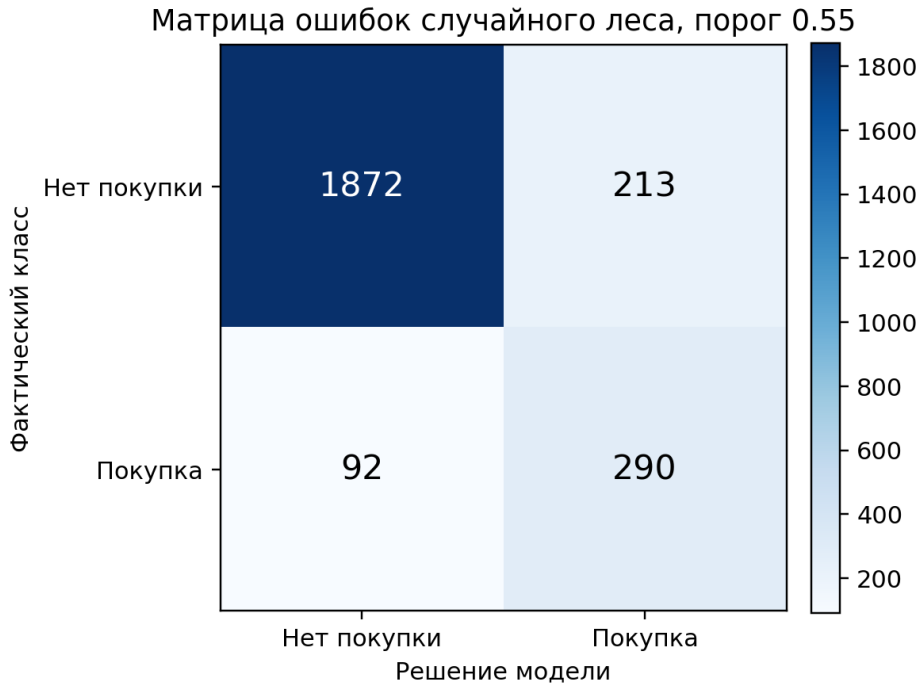


Рис. 7. Матрица ошибок выбранного классификатора на test

В матрице ошибок строки соответствуют фактическому классу, столбцы — решению модели. Главная диагональ содержит правильные ответы, а две

внедиагональные ячейки показывают разные типы ошибок. Левая нижняя ячейка — пропущенные покупки FN, правая верхняя — ложные предложения FP. Такая визуализация показывает абсолютный масштаб ошибок, который одна агрегированная метрика скрывает.

4.8. Кривые качества

Кривые строятся по небольшим локальным таблицам score и label, тогда как обучение и получение предсказаний остаются в Spark. Каждая точка кривой соответствует своему порогу: при его снижении модель обычно находит больше покупок, но одновременно увеличивает число ложных положительных решений.

```
# Графики классификации строятся по небольшой таблице valid/test.
fig, ax = plt.subplots(figsize=(7.5, 4.8))
for name, color in [("Логистическая регрессия", "tab:blue"),
                    ("Случайный лес", "tab:red")]:
    local = shop_valid_predictions[name].toPandas()
    p, r, _ = precision_recall_curve(local["label"], local["score"])
    ax.plot(r, p, label=name, color=color, linewidth=2)
ax.axhline(valid_pd["label"].mean(), linestyle="--", color="gray",
           label="Доля положительного класса")
ax.set(xlabel="Recall", ylabel="Precision",
       title="Precision–Recall-кривые на valid")
ax.grid(alpha=0.25)
ax.legend()
plt.tight_layout()
plt.savefig(OUT / "classification_pr.png", dpi=160)
plt.show()

final_pd = shop_test_scores.toPandas()
final_pred = (final_pd["score"] >= SHOP_THRESHOLD).astype(int)
cm = confusion_matrix(final_pd["label"].astype(int), final_pred, labels=[0, 1])
fig, ax = plt.subplots(figsize=(5, 4))
im = ax.imshow(cm, cmap="Blues")
for i in range(2):
    for j in range(2):
        ax.text(j, i, cm[i, j], ha="center", va="center", fontsize=13)
ax.set_xticks([0, 1], ["Нет покупки", "Покупка"])
ax.set_yticks([0, 1], ["Нет покупки", "Покупка"])
ax.set(xlabel="Решение модели", ylabel="Факт", title="Матрица ошибок на test")
fig.colorbar(im, ax=ax)
plt.tight_layout()
plt.savefig(OUT / "classification_confusion.png", dpi=160)
plt.show()
```

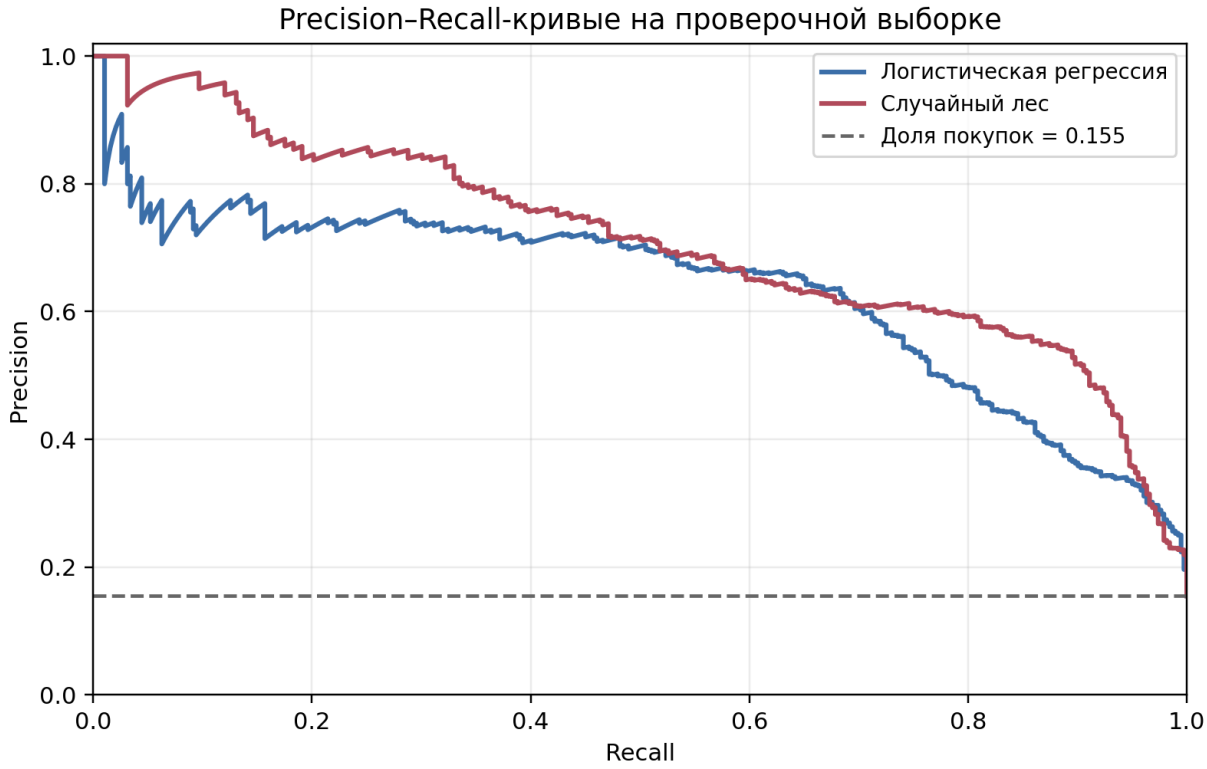


Рис. 8. Precision–Recall-кривые на valid

На Precision–Recall-кривой по горизонтали отложен recall, по вертикали — precision. Горизонтальная пунктирная линия равна доле покупок в valid и показывает качество случайного ранжирования без полезного сигнала. Чем дольше кривая сохраняет высокую precision при росте recall, тем лучше модель ранжирует редкий положительный класс.

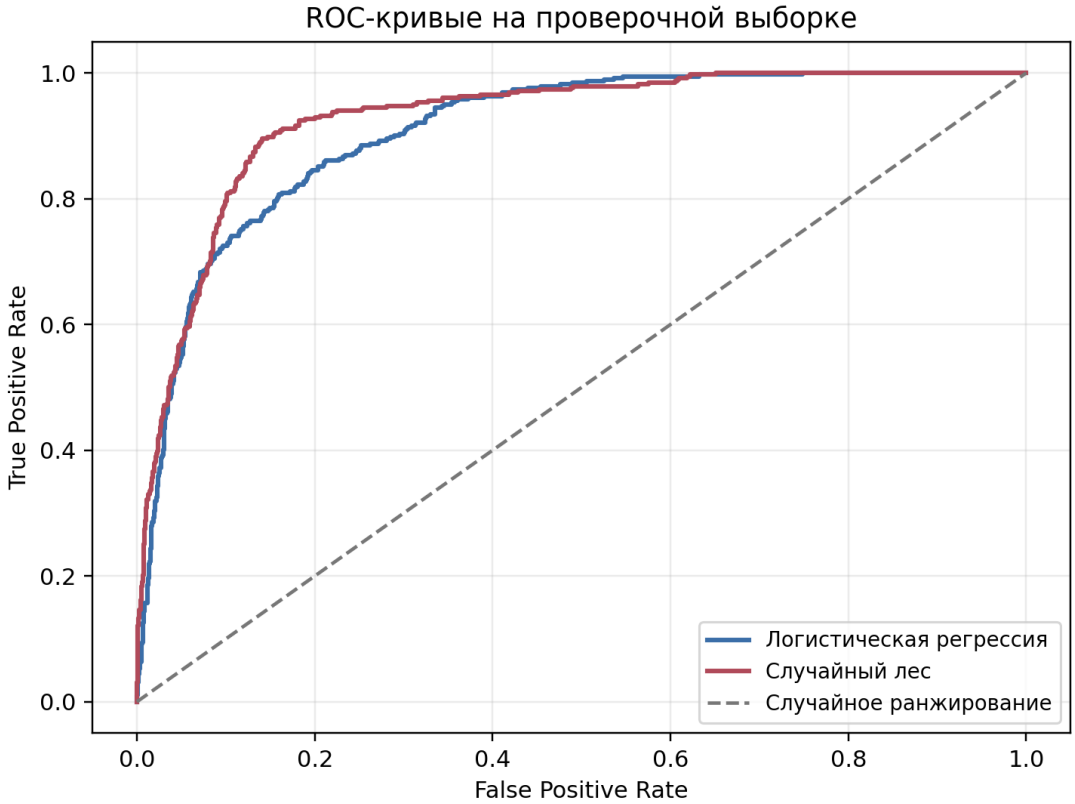


Рис. 9. ROC-кривые на valid

ROC-кривая сопоставляет true positive rate и false positive rate. Диагональ соответствует случайному ранжированию, а изгиб к левому верхнему углу — лучшему разделению классов. При сильном дисбалансе ROC-AUC может выглядеть высокой даже тогда, когда абсолютное число FP велико, поэтому график читается вместе с PR-кривой и матрицей ошибок.

Обе визуализации подтверждают, что случайный лес в среднем ранжирует сессии лучше логистической регрессии. Однако кривые не выбирают прикладной порог автоматически и не показывают калибровку вероятностей. Рабочая граница решения всё равно определяется на valid по заранее заданному правилу.

4.9. Важность признаков и абляция PageValues

Важность случайного леса показывает, насколько активно признак использовался для уменьшения ошибки в деревьях. Это не является оценкой причинного влияния и не гарантия устойчивости на другом сайте.

Важности леса относятся к координатам подготовленного вектора. Одна исходная категория после one-hot-кодирования представлена несколькими координатами, а взаимосвязанные признаки могут делить важность между собой. Кроме того, impurity-based importance склонна чаще использовать признаки с большим числом возможных разбиений. Поэтому диаграмма подходит для диагностики модели, но не для доказательства причинного влияния.

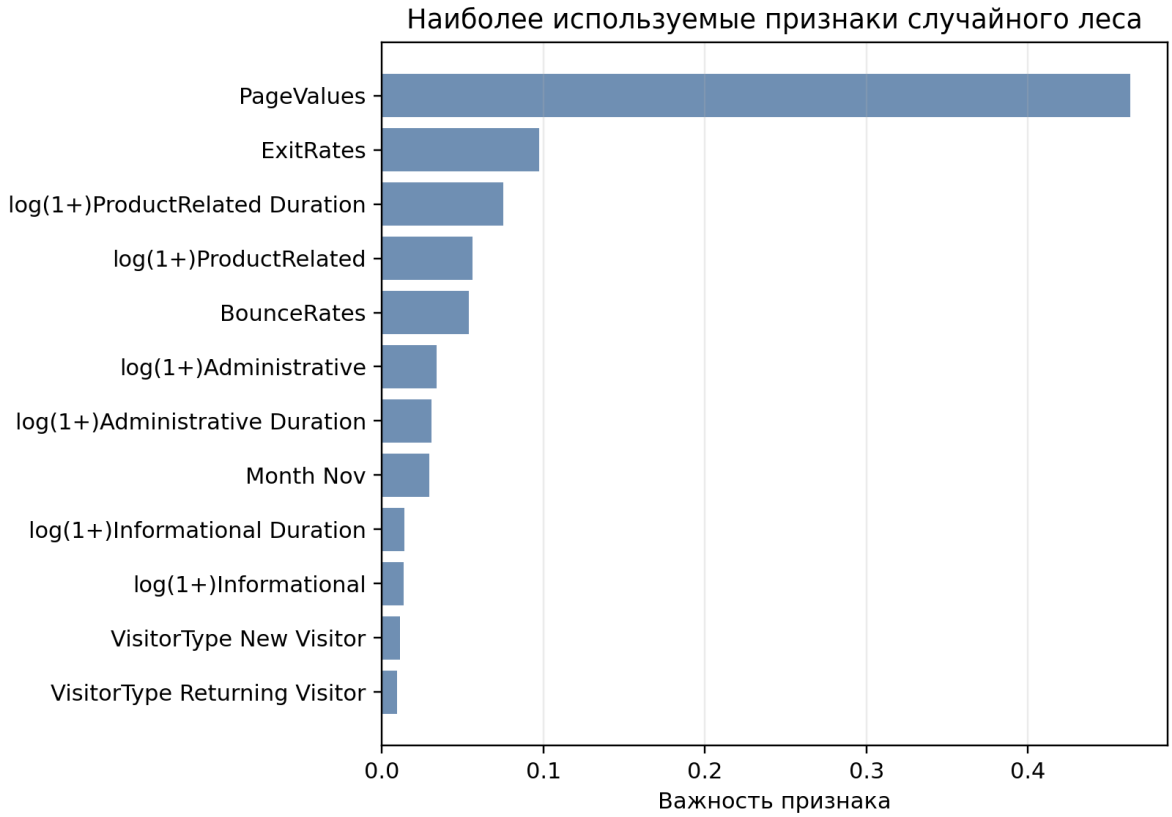


Рис. 10. Наиболее важные координаты случайного леса

На диаграмме PageValues заметно превосходит остальные координаты. Далее следуют ExitRates и показатели длительности товарных просмотров. Результат объясняет, почему отдельно проверяется вариант без PageValues. Если это поле недоступно в момент реального прогноза, высокая итоговая метрика основного опыта не может быть перенесена на ранний сценарий.

В анализе чувствительности модели будем менять только наличие PageValues. Состав train/valid, параметры случайного леса и остальные преобразования остаются прежними. Такой контроль позволяет связать изменение метрики именно с исключением поля, а не со случайным новым разбиением или другой конфигурацией модели.

```
# Проверка чувствительности к PageValues на той же valid.
rf_without_page_values = shop_pipeline("RF",
include_page_values=False).fit(shop_train)
without_scores = binary_scores(rf_without_page_values, shop_valid)
print("RF с PageValues:",
      binary_metrics(shop_valid_predictions["Случайный лес"],
ranking=True)["areaUnderPR"])
print("RF без PageValues:",
      binary_metrics(without_scores, ranking=True)["areaUnderPR"])
```

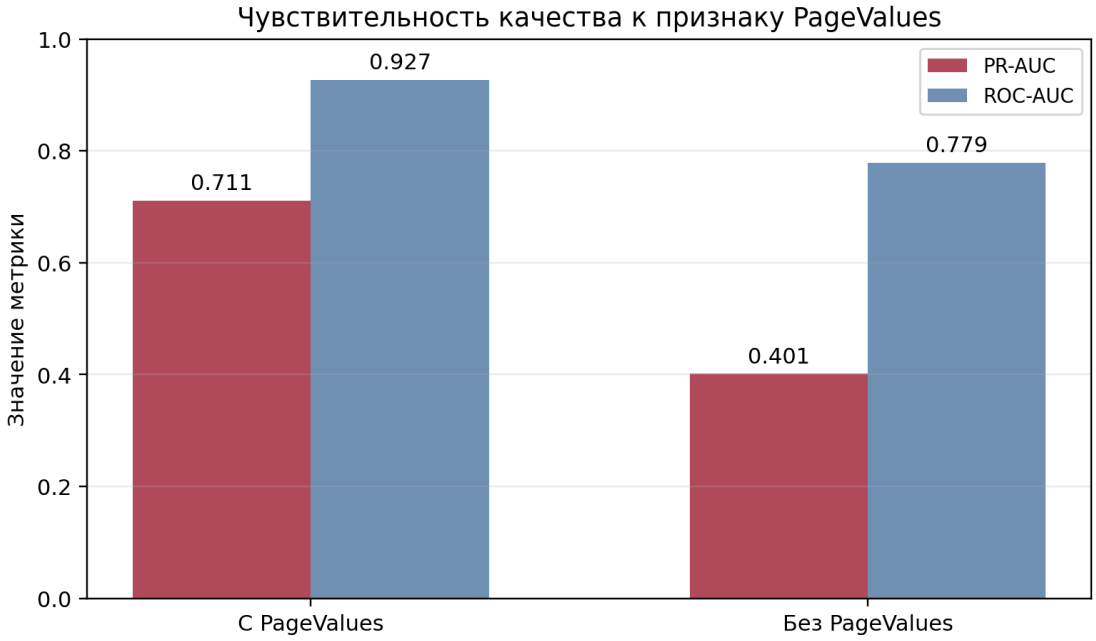


Рис. 11. Изменение качества после исключения PageValues

Таблица 6. Влияние признака PageValues для случайного леса

Вариант	ROC-AUC valid	PR-AUC valid
С PageValues	0,927	0,711
Без PageValues	0,779	0,401

После исключения PageValues контрольные PR-AUC и ROC-AUC заметно уменьшаются. График и таблица показывают зависимость решения от выбранного момента наблюдения. Для раннего прогноза необходимо принять более низкое качество, создать иные доступные признаки либо изменить постановку задачи; переносить прежние числа без новой проверки нельзя.

> Резкое падение PR-AUC означает, что итог сильно зависит от PageValues. Перед внедрением необходимо проверить, когда и как рассчитывается этот показатель. Нельзя переносить результат в сценарий раннего прогноза без новой проверки.

5. Регрессия спроса Seoul Bike

5.1. Постановка задачи

Цель — спрогнозировать число аренд за конкретный час. Ошибка на 20 и ошибка на 500 аренд имеют разный масштаб, поэтому основными метриками служат MAE и RMSE. В отличие от классификации, ответ не переводится в класс порогом.

Одна строка Seoul Bike описывает наблюдаемый час. В основной постановке удаляются часы Functioning Day = No, так как в них нулевое число аренд вызвано закрытием сервиса, а не обычным сочетанием спроса и погоды. Модель отвечает на вопрос о спросе при работающем прокате.

Нерабочие часы исключаются до обучения. Дата разделяется хронологически: ранние месяцы формируют train, август–сентябрь — valid, октябрь–ноябрь — test. Поздние часы не используются для обучения модели, которая проверяется на более раннем периоде.

Используемая погода фактически наблюдалась в соответствующий час. Поэтому опыт является ретроспективной оценкой зависимости спроса от календаря и погодных условий. Для оперативного прогноза будущего часа потребовались бы погодные прогнозы, доступные в момент выдачи результата, и новая независимая проверка качества.

5.2. Временные признаки и части выборки

Час суток является циклическим: 23:00 и 00:00 соседствуют. Поэтому создаются hour_sin и hour_cos. Аналогично кодируется месяц. day_index хранит число дней от начала набора и используется в лесной модели как простой индикатор общего временного сдвига.

Пара sin/cos сохраняет близость конца и начала цикла: часы 23 и 0 оказываются рядом на окружности, а не на противоположных концах числовой шкалы. Одной координаты недостаточно, потому что одинаковое значение синуса встречается в разные часы. Для месяца применяется то же представление с периодом 12.

$$hour_sin = sin(2\pi \cdot hour / 24), \quad hour_cos = cos(2\pi \cdot hour / 24)$$

$$month_sin = sin(2\pi \cdot month / 12), \quad month_cos = cos(2\pi \cdot month / 12)$$

Код сначала преобразует текстовую дату в тип date и исключает закрытые часы. Затем создаются день недели, циклические координаты часа и месяца, а day_index измеряет число дней от начала наблюдений. Последний признак используется только в лесной модели как простой индикатор общего временного сдвига.

Разбиение выполняется фильтрами по дате: ранний период образует train, август–сентябрь — valid, октябрь–ноябрь — test. count() материализует кэш, min/max позволяют визуально проверить границы, а assert подтверждает, что после фильтрации потерянных строк нет. Ни среднее target, ни статистики test при этом не участвуют в обучении.

```
bike = (bike_raw
    .withColumn("date", F.to_date("date_text", "dd/MM/yyyy"))
    .drop("date_text")
    .filter(F.col("functioning_day") == "Yes")
    .withColumn("weekday", F.dayofweek("date").cast("string"))
```

```
    .withColumn("hour_sin",
                F.sin(F.lit(2.0 * math.pi) * F.col("hour") / F.lit(24.0)))
    .withColumn("hour_cos",
                F.cos(F.lit(2.0 * math.pi) * F.col("hour") / F.lit(24.0)))
    .withColumn("month_sin",
                F.sin(F.lit(2.0 * math.pi) * F.month("date") / F.lit(12.0)))
    .withColumn("month_cos",
                F.cos(F.lit(2.0 * math.pi) * F.month("date") / F.lit(12.0)))
    .withColumn("day_index",
                F.datediff("date", F.lit("2017-12-
01").cast("date")).cast("double")))

bike_train = bike.filter("date < DATE '2018-08-01'").cache()
bike_valid = bike.filter(
    "date >= DATE '2018-08-01' AND date < DATE '2018-10-01'"
).cache()
bike_test = bike.filter("date >= DATE '2018-10-01'").cache()

for name, df in [("train", bike_train), ("valid", bike_valid), ("test",
bike_test)]:
    print(name, df.count())
    df.agg(F.min("date"), F.max("date")).show()
```

Таблица 7. Временное разбиение Seoul Bike после исключения закрытых часов

Часть	Календарный интервал	Строк
Train	01.12.2017–31.07.2018	5 784
Valid	01.08.2018–30.09.2018	1 368
Test	01.10.2018–30.11.2018	1 313

Таблица фиксирует непересекающиеся временные интервалы и число строк в каждом из них. Valid отражает более поздний сезон, чем большая часть train, а test расположен ещё дальше во времени. Поэтому разрыв метрик может быть связан не только с переобучением, но и с изменением уровня спроса и погоды между периодами.

5.3. Три регрессионных решения

Линейная регрессия строит взвешенную сумму подготовленных признаков. Она объяснима, но не обязана отражать сложный суточный профиль спроса.

$$\hat{y} = b + \sum_{j} w_j x_j$$

Линейность относится к уже подготовленному вектору признаков, в котором категориальные признаки представлены индикаторами, а час и месяц — координатами окружности. Коэффициенты оценивают добавочный вклад координат при фиксированных остальных признаках. L2-регуляризация ограничивает слишком большие веса и делает решение устойчивее при взаимосвязанных полях.

Полиномиальная регрессия добавляет квадраты и попарные произведения пяти признаков: температуры, влажности, солнечной радиации и двух координат часа. Модель остаётся линейной по коэффициентам, но зависимость от исходных полей становится нелинейной:

$$\hat{y} = b + w_1x_1 + w_2x_2 + w_3{x_1}^2 + w_4x_1x_2 + w_5{x_2}^2.$$

В реализации степень 2 применяется только к пяти координатам: temperature, humidity, solar_radiation, hour_sin и hour_cos. Получается 20 слагаемых без константы. Остальные числовые и категориальные признаки добавляются после расширения. Повышение степени увеличивает размерность и риск нестабильности, поэтому полиномиальная ветка получает более сильную регуляризацию.

Случайный лес строит нелинейную кусочно-постоянную зависимость и учитывает взаимодействия без ручного перечисления произведений. В его вектор дополнительно включается day_index. Это помогает на данном годовом отрезке, но дерево не умеет свободно продолжать временной тренд далеко за пределы обученных дат.

Регрессионное дерево делит пространство признаков на области и возвращает среднее целевое значение обучающих строк в соответствующем листе. Случайный лес усредняет ответы 120 таких деревьев глубины не более 12, минимум пять наблюдений в узле и случайная доля 0,7 признаков ограничивают чрезмерную подстройку. Лес хорошо моделирует взаимодействия внутри наблюдаемого диапазона, но он не предназначен для свободной экстраполяции возрастающего временного тренда.

Практический блок 5.4. Pipeline регрессоров

Функция bike_pipeline создаёт новый набор этапов для каждого вида модели. StringIndexer и OneHotEncoder преобразуют season, holiday и weekday. Для линейной ветки VectorAssembler объединяет погодные, циклические и категориальные координаты, StandardScaler обучается на train, после чего LinearRegression оценивает коэффициенты.

В полиномиальной ветке сначала собираются пять выбранных координат, PolynomialExpansion добавляет квадраты и взаимодействия, затем к ним присоединяются остальные признаки и выполняется масштабирование. В ветке обучения алгоритма случайного леса используется исходный числовой набор плюс day_index. В данном обучении scaler не нужен, потому что деревья работают с пороговыми разбиениями.

Все три решения обучаются на одинаковом bike_train и проверяются на одном bike_valid, поэтому сравниваются готовые цепочки предобработки и модели.

Различие метрик отражает одновременно класс алгоритма, представление признаков и заданную регуляризацию.

```
BIKE_CAT = ["season", "holiday", "weekday"]
BIKE_NUM = [
    "temperature", "humidity", "wind_speed", "visibility",
    "dew_point", "solar_radiation", "rainfall", "snowfall",
    "hour_sin", "hour_cos", "month_sin", "month_cos"
]
BIKE_RF_NUM = BIKE_NUM + ["day_index"]
POLY_SOURCE = ["temperature", "humidity", "solar_radiation", "hour_sin",
"hour_cos"]
POLY_REST = [c for c in BIKE_NUM if c not in POLY_SOURCE]


def bike_pipeline(kind):
    index_cols = [c + "_idx" for c in BIKE_CAT]
    encoded_cols = [c + "_ohe" for c in BIKE_CAT]
    indexer = StringIndexer(
        inputCols=BIKE_CAT, outputCols=index_cols,
        handleInvalid="keep", stringOrderType="alphabetAsc"
    )
    encoder = OneHotEncoder(
        inputCols=index_cols, outputCols=encoded_cols,
        handleInvalid="keep", dropLast=False
    )
    stages = [indexer, encoder]

    if kind == "LINEAR":
        stages += [
            VectorAssembler(
                inputCols=BIKE_NUM + encoded_cols, outputCol="features_raw"
            ),
            StandardScaler(
                inputCol="features_raw", outputCol="features",
                withMean=False, withStd=True
            ),
            LinearRegression(
                maxIter=150, regParam=0.1, elasticNetParam=0.0,
                standardization=False
            )
        ]
    elif kind == "POLY":
        stages += [
            VectorAssembler(inputCols=POLY_SOURCE, outputCol="poly_input"),
            PolynomialExpansion(
                degree=2, inputCol="poly_input", outputCol="poly_features"
            ),
            VectorAssembler(
                inputCols=["poly_features"] + POLY_REST + encoded_cols,
                outputCol="features_raw"
            ),
            StandardScaler(
                inputCol="features_raw", outputCol="features",
                withMean=False, withStd=True
            ),
            LinearRegression(
                maxIter=200, regParam=1.0, elasticNetParam=0.0,
                standardization=False
            )
        ]
    elif kind == "RF":
```

```
        stages += [
            VectorAssembler(
                inputCols=BIKE_RF_NUM + encoded_cols, outputCol="features"
            ),
            RandomForestRegressor(
                numTrees=120, maxDepth=12, maxBins=64,
                minInstancesPerNode=5, featureSubsetStrategy="0.7",
                seed=SEED
            )
        ]
    else:
        raise ValueError(kind)
    return Pipeline(stages=stages)
```

Вызов bike_pipeline(kind) только описывает вычислительный граф. Реальные словари категорий, стандартные отклонения и параметры регрессора появляются при fit(bike_train). Полученный PipelineModel затем применяется к более поздним периодам без повторного обучения преобразований.

> Сравниваются полные решения «предобработка + модель». У линейной, полиномиальной и ветви с обучением случайного леса представление признаков различается осознанно. Поэтому результат нельзя трактовать как чистое сравнение одной математической формулы при абсолютно одинаковом входе.

5.5. Метрики и базовые правила

MAE сохраняет единицу целевой переменной и показывает средний модуль ошибки в числе аренд. RMSE также измеряется в арендах, но квадрат ошибки делает крупные промахи заметно более весомыми. Поэтому RMSE полезна, когда особенно нежелательны ошибки на часах пикового спроса.

$$MAE = \left(\frac{1}{n}\right)\Sigma_i |y_i - \hat{y}_i|, \quad RMSE = \sqrt{\left[\left(\frac{1}{n}\right)\Sigma_i (y_i - \hat{y}_i)^2\right]}$$

$$R^2 = 1 - \frac{\Sigma_i (y_i - \hat{y}_i)^2}{\Sigma_i (y_i - \bar{y})^2}$$

R^2 сравнивает сумму квадратов ошибок модели с разбросом фактических значений вокруг среднего оцениваемой части. Это не процент правильных прогнозов. На новых данных R^2 может быть отрицательным, если модель хуже постоянного прогноза, равного среднему факту этой части.

Функция regression_metrics последовательно вызывает RegressionEvaluator для трёх метрик и дополнительно считает долю отрицательных прогнозов. Линейная модель не содержит естественного ограничения prediction ≥ 0, поэтому такие

случаи не обрезаются. Возможное правило max(0, prediction) следовало бы выбирать и оценивать отдельно на valid.

```
def regression_metrics(pred):
    result = {}
    for metric in ["rmse", "mae", "r2"]:
        result[metric] = RegressionEvaluator(
            labelCol="label", predictionCol="prediction", metricName=metric
        ).evaluate(pred)
    result["negative_share"] = pred.agg(
        F.avg((F.col("prediction") < 0).cast("double"))
    ).first()[0]
    return result

train_mean = bike_train.agg(F.avg("label")).first()[0]
hour_means = bike_train.groupBy("hour").agg(
    F.avg("label").alias("hour_mean")
)

def bike_baseline(df, by_hour=False):
    if not by_hour:
        return df.withColumn("prediction", F.lit(train_mean))
    return (df.join(hour_means, "hour", "left")
        .withColumn("prediction", F.coalesce("hour_mean", F.lit(train_mean))))
```

Первое базовое правило всегда возвращает среднее train. Второе использует среднее число аренд для каждого часа суток, рассчитанное только по train. Простое почасовое правило уже учитывает суточный ритм и является содержательным соперником.

Среднее по часу — более сильный baseline, чем одна глобальная константа: оно использует только train, но уже воспроизводит суточный профиль спроса. Если сложная модель не превосходит это правило на valid, её дополнительная вычислительная сложность пока не подтверждена. Средние из valid и test в baseline не попадают.

5.6. Обучение и сравнение

Цикл последовательно обучает три Pipeline на bike_train, измеряет время fit и материализует предсказания valid содержательной агрегацией. В таблицу добавляются два заранее определённых базовых правила. Лучшая обучаемая модель выбирается по минимальной RMSE valid. Только после фиксации имени она один раз применяется к bike_test.

```
bike_models = {}
bike_rows = []
bike_valid_predictions = {}

for name, kind in [("Линейная регрессия", "LINEAR"),
                   ("Полиномиальная регрессия", "POLY"),
                   ("Случайный лес", "RF")]:
    sc.setJobGroup("P6_REG_" + kind, "Регрессия: " + name)
```

```
    t0 = perf_counter()
    model = bike_pipeline(kind).fit(bike_train)
    fit_s = perf_counter() - t0
    pred = model.transform(bike_valid).cache()
    pred.select(F.avg("prediction")).first()
    bike_models[name] = model
    bike_valid_predictions[name] = pred
    bike_rows.append({"model": name, "fit_s": fit_s,
**regression_metrics(pred)})

for name, by_hour in [("Среднее train", False), ("Среднее по часу", True)]:
    bike_rows.append({"model": name,
**regression_metrics(bike_baseline(bike_valid, by_hour))})

bike_validation = pd.DataFrame(bike_rows).sort_values("rmse")
print(bike_validation.round(3).to_string(index=False))

learned = bike_validation[bike_validation["model"].isin(bike_models)]
BEST_BIKE_NAME = str(learned.iloc[0]["model"])
BEST_BIKE = bike_models[BEST_BIKE_NAME]
print("Выбрана модель:", BEST_BIKE_NAME)

bike_test_prediction = (BEST_BIKE.transform(bike_test)
    .select("date", "hour", "label", "prediction")
    .withColumn("residual", F.col("label") - F.col("prediction"))
    .cache())
print("TEST:", regression_metrics(bike_test_prediction))
```

Таблица 8. Контрольные метрики регрессии

Решение	RMSE valid	MAE valid	R^2 valid	RMSE test	MAE test	R^2 test
Среднее train	717.4	539.7	-0.256	597.7	463.5	-0.134
Среднее по часу	559.1	429.7	0.237	439.8	323.6	0.386
Линейная регрессия	526.4	422.0	0.323	542.6	402.5	0.065
Полиномиальная регрессия	415.0	307.4	0.580	384.8	257.6	0.530
Случайный лес	288.2	200.4	0.797	316.5	208.6	0.682

Контрольная таблица позволяет сравнить не только RMSE, но и MAE, R^2 и долю отрицательных прогнозов. Полиномиальная модель выигрывает у обычной линейной, значит квадраты и взаимодействия действительно описывают часть нелинейности. Случайный лес получает лучшую RMSE среди обучаемых решений, но окончательный вывод ограничен выбранным временным разбиением.

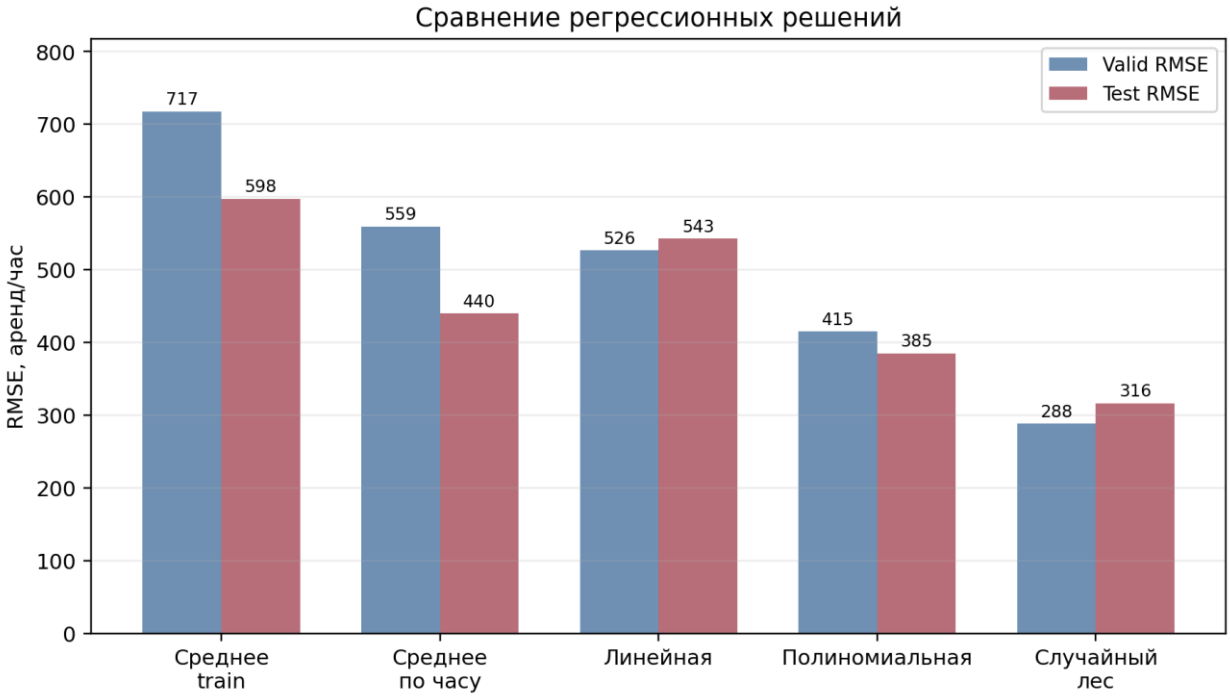


Рис. 7. RMSE базовых правил и трёх моделей

Столбчатая диаграмма сопоставляет RMSE решений на valid и test. Небольшой разрыв между периодами говорит об относительной устойчивости, а сильный рост test-ошибки предупреждает о переносе на более поздний сезон. На графике видно, что почасовое среднее значительно сильнее глобального среднего, полиномиальная регрессия лучше линейной, а случайный лес сохраняет наименьшую ошибку.

В контрольном запуске случайный лес получил RMSE около 288 на valid и 316 на test. Полиномиальная модель заметно лучше обычной линейной, что согласуется с нелинейной зависимостью спроса от времени суток и погоды. Почасовое среднее превосходит глобальное среднее и показывает, почему модель необходимо сравнивать с содержательным простым правилом.

5.7. Проверка итоговой модели

Для визуализаций в pandas передается только подвыборка тестовых строк с фактом, прогнозом, датой и часом. Из даты и часа собирается временная метка. Первый график сравнивает каждое фактическое значение с прогнозом, второй показывает последовательность одной недели. Дополнительная агрегация по часу суток используется для анализа средней абсолютной ошибки.

```
# Графики регрессии строятся по тестовой таблице
reg_local = bike_test_prediction.orderBy(«date», «hour»).toPandas()
reg_local[«timestamp»] = (
    pd.to_datetime(reg_local[«date»]) +
    pd.to_timedelta(reg_local[«hour»], unit=»h»)
)
```

```
fig, ax = plt.subplots(figsize=(6.4, 5.0))
ax.scatter(reg_local[«label»], reg_local[«prediction»], s=12, alpha=0.35)
limit = max(reg_local[«label»].max(), reg_local[«prediction»].max())
ax.plot([0, limit], [0, limit], «—«, color=»tab:red»)
ax.set(xlabel=»Фактическое число аренд», ylabel=»Прогноз»,
       title=»Факт и прогноз на test»)
ax.grid(alpha=0.2)
plt.tight_layout()
plt.savefig(OUT / «regression_scatter.png», dpi=160)
plt.show()

week = reg_local[reg_local[«timestamp»] < reg_local[«timestamp»].min() +
pd.Timedelta(days=7)]
fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(week[«timestamp»], week[«label»], label=»Факт»)
ax.plot(week[«timestamp»], week[«prediction»], label=»Прогноз»)
ax.set(xlabel=»Дата и час», ylabel=»Число аренд»,
       title=»Первая неделя тестового периода»)
ax.grid(alpha=0.2)
ax.legend()
plt.tight_layout()
plt.savefig(OUT / «regression_week.png», dpi=160)
plt.show()
```

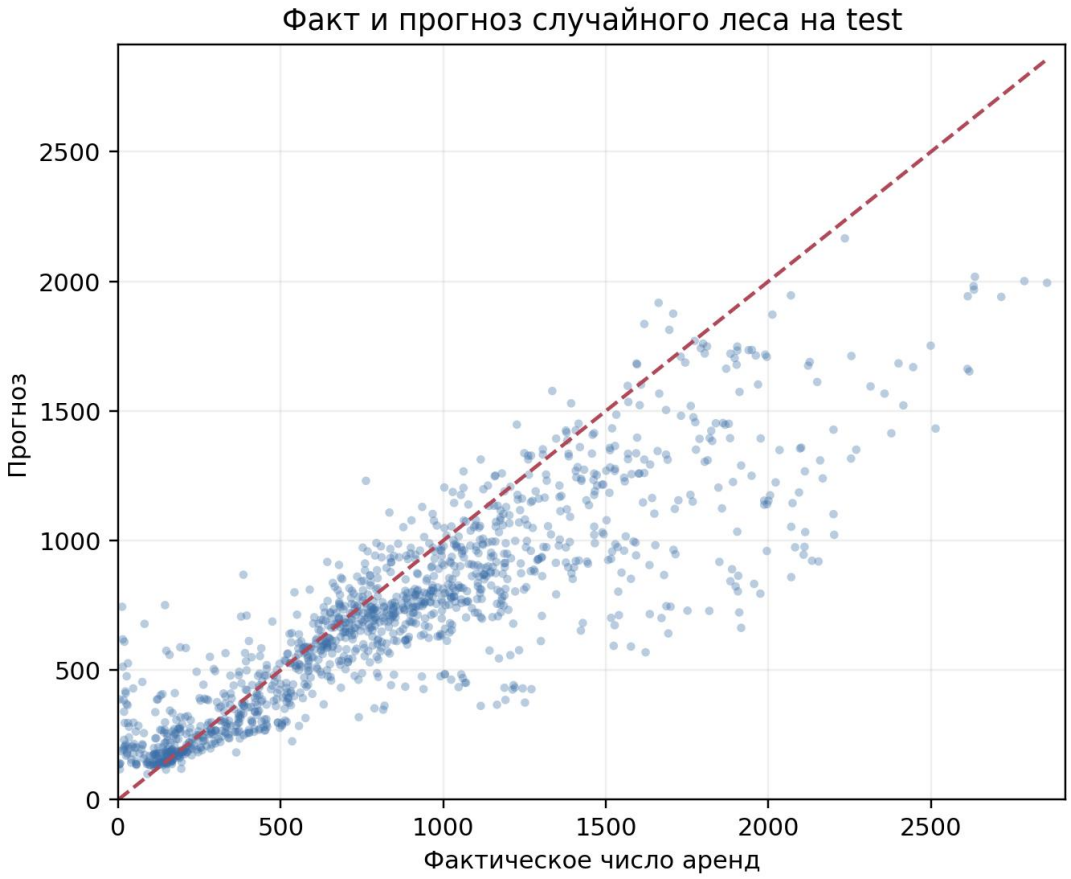


Рис. 8. Фактическое и предсказанное число аренд на test

Пунктирная диагональ соответствует идеальному прогнозу $y = \hat{y}$. Точки ниже диагонали означают занижение, выше — завышение. Разброс возрастает вместе со спросом, малые и средние значения восстанавливаются точнее, а крупные пики

случайный лес систематически сглаживает. Такая форма ошибки не видна из одной RMSE. Она может сигнализировать о гетероскедастичности, более сложной модели сезонности чем обычное сложение.



Рис. 9. Факт и прогноз в течение первой недели test

Линейчатый график сохраняет временной порядок дат и показывает, насколько прогноз воспроизводит суточные подъёмы и спады. Совпадение формы волн означает, что модель уловила ритм, а различие высоты пиков показывает амплитудную ошибку. Отдельно нужно искать временные сдвиги, так как даже правильная величина, предсказанная на соседний час, остаётся ошибкой.



Рис. 10. Средняя абсолютная ошибка по часу суток

Группировка абсолютной ошибки по часу суток раскрывает неоднородность качества. Наибольшие значения приходятся на часы резкого изменения спроса, прежде всего утренние и вечерние пики. Ночные часы проще, поскольку спрос там ниже и стабильнее. Такой график подсказывает, где нужны дополнительные признаки или отдельная модель режима пиковых нагрузок.

Вместе три графика отвечают на разные вопросы. График разброса scatter показывает форму и масштаб ошибки, недельный ряд — сохранение временной динамики, а профиль MAE по часу — устойчивые проблемные интервалы. Ни одна визуализация отдельно не устанавливает причину ошибки, но их согласованный анализ помогает сформулировать проверяемые гипотезы.

5.8. Важность признаков и ограничения

Для регрессии случайным лесом важность координат извлекается из последнего этапа Pipeline и сопоставляется с именами подготовленного вектора. Диаграмма показывает, какие признаки чаще уменьшали неоднородность целевой переменной в узлах деревьев. Она не показывает направление влияния и может распределять вклад между взаимосвязанными признаками.

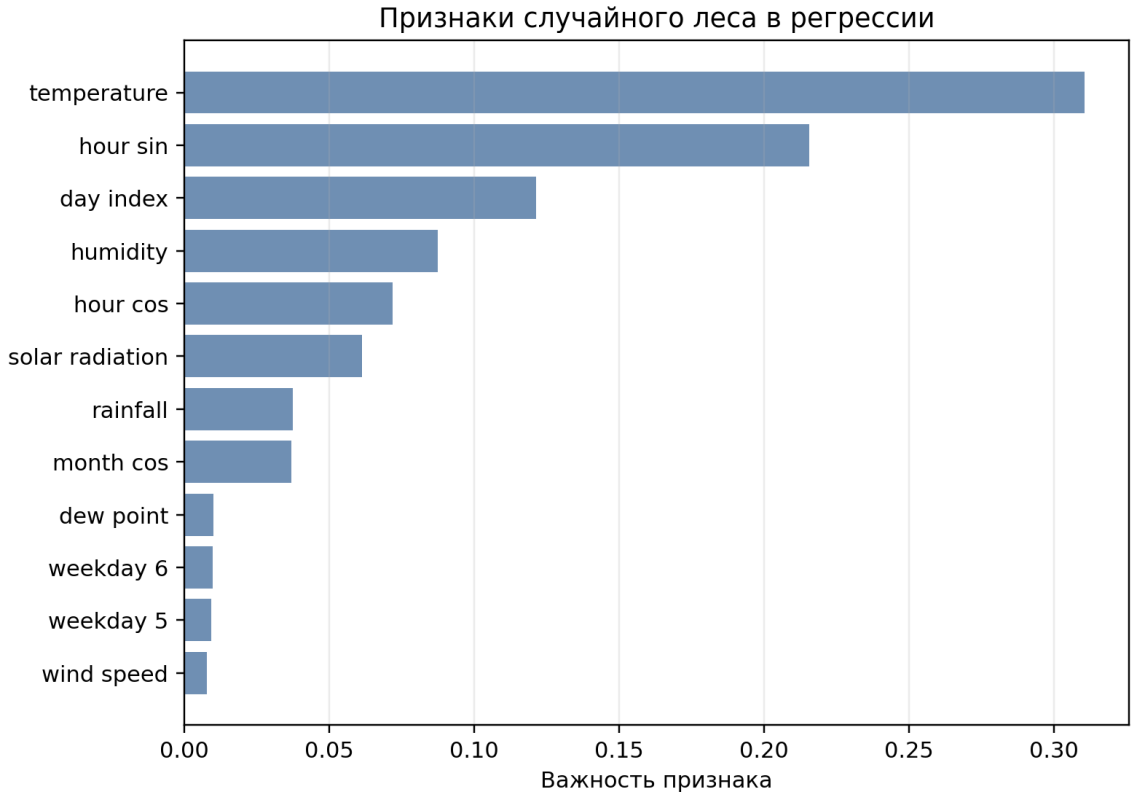


Рис. 11. Наиболее используемые признаки лесной регрессии

Температура и циклические координаты часа занимают верхние позиции, что согласуется с сезонным и суточным характером спроса. Высокая важность day_index означает, что модель активно использует положение часа во времени.

Это улучшает качество внутри одного года, но делает обязательной отдельную проверку при переносе на следующий период, где продолжение тренда может измениться.
Температура, положение часа на окружности и day_index используются чаще остальных. Нельзя делать вывод, что повышение температуры само по себе вызовет рост аренды, так как модель описывает совместные закономерности предикторов в наблюдательных данных. Важность day_index также предупреждает, что перенос на другой год требует отдельной проверки.

> Линейные модели иногда возвращают отрицательное число аренд. В контрольном test доля таких ответов составляет около 7,6% для обычной линейной регрессии и 1,2% для полиномиальной. Нельзя заменить их нулём и оставить прежние метрики, так как обрезка является новым правилом и должно выбираться на валидационной выборке.

6. Кластеризация интернет-сессий

6.1. Смысл задачи

В кластеризации нет правильной метки группы. Алгоритм ищет сессии, которые близки по выбранным признакам. Номер кластера — технический идентификатор, поэтому кластер 0 не обязан быть лучше или хуже кластера 1.
Вектор содержит количество и длительность административных, информационных и товарных просмотров, а также BounceRates и ExitRates. Целевая метка Revenue и спорный признак PageValues не участвуют. Revenue подключается только после выбора k, чтобы описать связь найденных групп с покупкой.
Выбор признаков задаёт само понятие похожести. Длительности и количества просмотров имеют длинный правый хвост, поэтому для шести полей используется log1p. После этого все восемь координат центрируются и делятся на стандартное отклонение. Без масштабирования признаки с большим числовым диапазоном почти полностью определяли бы евклидово расстояние.
Масштабирование не делает признаки независимыми. Количество страниц и время на них связаны, а BounceRates и ExitRates частично описывают близкое поведение. Повторение похожего сигнала увеличивает его вес в геометрии. Поэтому набор координат выбирается содержательно без принципа включения любых признаков без разбора.

Таблица 9. Признаки кластеризации

Группа	Преобразование
6 счётчиков и длительностей	log1p для уменьшения правого хвоста
BounceRates, ExitRates	без логарифма
Все 8 координат	центрирование и деление на стандартное отклонение
Revenue, PageValues	не входят в расстояние

Итоговый вектор имеет восемь координат, среди которых шесть логарифмированных счётчиков и длительностей плюс BounceRates и ExitRates. Revenue исключён как целевой результат покупки, а PageValues — чтобы сегментация описывала поведение сессии, а не строилась непосредственно вокруг денежного результата.

Практический блок 6.2. Подготовка вектора

SQLTransformer создаёт логарифмированные копии исходных полей, VectorAssembler собирает их в cluster_raw, а StandardScaler обучает центр и масштаб каждой координаты. Здесь используется withMean=True: вектор небольшой и плотный, поэтому центрирование допустимо и полезно для методов расстояния.

Кластеризация в этом эксперименте описывает уже имеющийся набор, а не прогнозирует будущую метку. Поэтому scaler обучается на всех сессиях без Revenue. Для переноса на новый поток потребовалось бы обучить подготовку на историческом периоде, зафиксировать её и отдельно проверять изменение распределений.

Полный shop_vectors остаётся распределённым и используется для обучения KMeans. Отдельная детерминированная выборка из 4 000 строк создаётся до toPandas() и нужна только для вычисления силуэта и визуализации.

```
CLUSTER_LOG_SOURCE = [
    "Administrative", "Administrative_Duration",
    "Informational", "Informational_Duration",
    "ProductRelated", "ProductRelated_Duration"
]
CLUSTER_LOG = ["cluster_log_" + c for c in CLUSTER_LOG_SOURCE]
CLUSTER_RATE = ["BounceRates", "ExitRates"]

cluster_expressions = [
    f"log1p(`{source}`) AS `{target}`"
    for source, target in zip(CLUSTER_LOG_SOURCE, CLUSTER_LOG)
]
cluster_prep = Pipeline(stages=[
    SQLTransformer(
        statement="SELECT *, " + ", ".join(cluster_expressions) + " FROM
__THIS__"
    ),
    VectorAssembler(
        inputCols=CLUSTER_LOG + CLUSTER_RATE,
```

```
        outputCol="cluster_raw"
    ),
    StandardScaler(
        inputCol="cluster_raw", outputCol="features",
        withMean=True, withStd=True
    )
])

cluster_prep_model = cluster_prep.fit(shop_raw)
shop_vectors = cluster_prep_model.transform(shop_raw).select("row_id",
"features").cache()

sample_order = F.sha2(F.concat_ws(
    ":", F.col("row_id").cast("string"), F.lit(str(SEED))
), 256)
cluster_sample = shop_vectors.orderBy(sample_order,
"row_id").limit(4000).cache()
cluster_sample.count()
```

После fit подготовительная часть становится PipelineModel со статистиками масштабирования. Колонка row_id сохраняется только для соединения назначений с исходными строками и не входит в расстояние.

6.3. KMeans и выбор k

KMeans попеременно назначает каждую точку ближайшему центру и пересчитывает центры как средние. Число групп задаётся заранее. Сумма квадратов расстояний почти всегда уменьшается при росте k, поэтому выбирать k только по trainingCost нельзя.

Целевая функция KMeans минимизирует сумму квадратов расстояний от каждой точки до центра назначенного ей кластера:

$$J = \Sigma_i \|x_i - \mu c(i)\|^2 \rightarrow min$$

Алгоритм чередует два шага. На первом назначает точки ближайшим центрам, а после пересчитывает центры как средние координат своих групп. Инициализация влияет на решение, поэтому фиксируется seed.

KMeans лучше всего описывает относительно компактные группы вокруг центров. Вытянутые структуры, выбросы и сильно различающаяся плотность могут ухудшить интерпретацию.

Для каждой точки метрика силуэта сравнивает среднее расстояние a до объектов своего кластера с минимальным средним расстоянием b до другого кластера:

$$s(i) = [b(i) - a(i)] / max[a(i), b(i)]$$

Значение ближе к 1 соответствует выраженному разделению, около 0 — объекту на границе, отрицательное — сомнительному назначению. Средний silhouette полезен для сравнения вариантов на одном наборе и в одной метрике, но не заменяет проверку размеров и профилей групп.

Цикл обучает KMeans для k от 2 до 8 на полном распределённом наборе. Для каждого варианта сохраняются trainingCost и минимальный размер кластера. Метрика Silhouette вычисляется на одной и той же выборке 4 000 строк обычным евклидовым расстоянием. Таблица сортируется по silhouette, а при близких значениях предпочтение меньшему k должно дополнительно подтверждаться интерпретируемыми профилями.

```
k_models = {}
k_rows = []

for k in range(2, 9):
    sc.setJobGroup("P6_CLUSTER_" + str(k), f"KMeans k={k}")
    model = KMeans(k=k, seed=SEED, maxIter=60, tol=1e-4).fit(shop_vectors)
    sample_pred = model.transform(cluster_sample).select("features",
"prediction")
    local = sample_pred.toPandas()
    x = np.vstack([v.toArray() for v in local["features"]])
    labels = local["prediction"].to_numpy()
    silhouette = silhouette_score(x, labels, metric="euclidean")

    sizes = model.transform(shop_vectors).groupBy("prediction").count()
    k_rows.append({
        "k": k,
        "silhouette": silhouette,
        "training_cost": model.summary.trainingCost,
        "min_cluster": sizes.agg(F.min("count")).first()[0]
    })
    k_models[k] = model

k_table = pd.DataFrame(k_rows).sort_values(
    ["silhouette", "k"], ascending=[False, True]
)
print(k_table.round(4).to_string(index=False))
BEST_K = int(k_table.iloc[0]["k"])
BEST_KMEANS = k_models[BEST_K]
print("Выбрано k =", BEST_K)
```

Таблица 10. Контрольный подбор числа кластеров

k	Silhouette	Within-cluster sum	Мин. кластер
2	0.324	66709	6134
3	0.444	42808	977
4	0.425	27942	878
5	0.380	23775	784
6	0.377	21077	782
7	0.344	18551	781
8	0.338	17022	578

В таблице trainingCost монотонно уменьшается при росте k, что ожидаемо, так как большему числу центров легче приблизиться к данным. Поэтому минимум стоимости не является критерием выбора. Одновременно проверяется min_cluster,

чтобы высокий силуэт не был получен ценой одной большой группы и нескольких почти пустых.

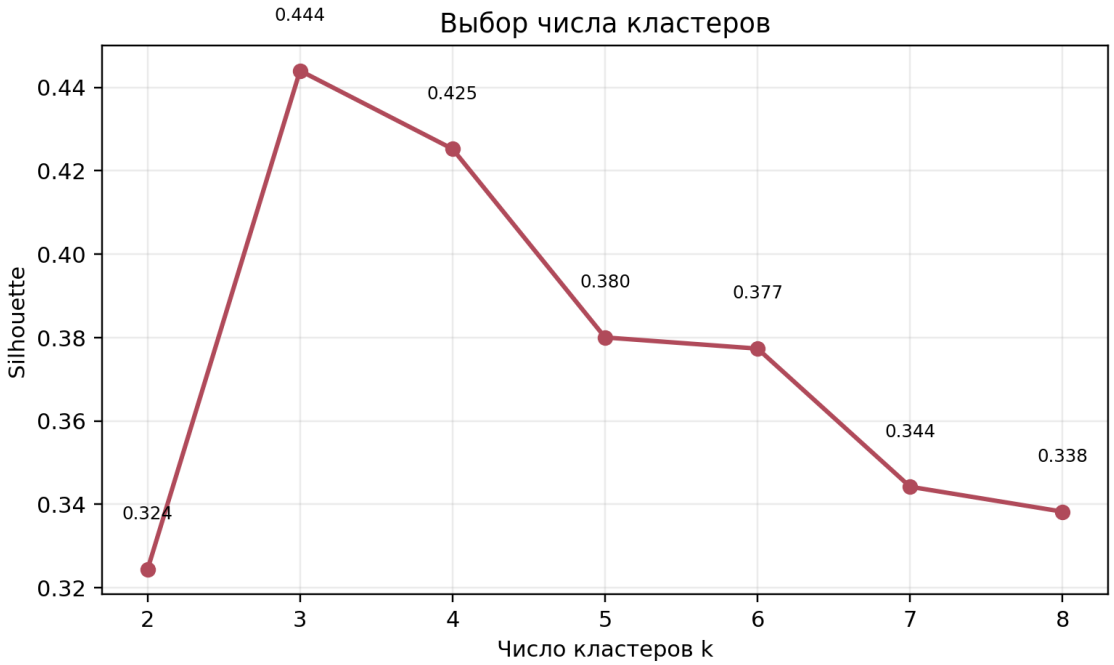


Рис. 12. Silhouette при разном числе кластеров

На графике видим, что заметный максимум при k = 3 сменяется постепенным снижением. Близкое значение k = 4 показывает, что выбор не абсолютно однозначен. Окончательное решение принимается после просмотра размеров и исходных характеристик кластеров, а не только по вершине линии.

6.4. Профили и двумерная проекция

После выбора k технические метки соединяются с исходными строками по row_id. Профиль строится в исходных единицах — числе страниц, длительности, BounceRates и ExitRates, — потому что значения стандартизованного вектора трудно интерпретировать предметно. Revenue добавляется только теперь как описательная характеристика и не влияет на построение групп.

Метод PCA, используемый только для визуализации, отдельно обучается на подготовленном восьмимерном пространстве и строит две линейные комбинации с наибольшей дисперсией. Двумерная проекция неизбежно теряет часть информации, поэтому перекрытие или разделение точек на рисунке не является строгим доказательством структуры в исходном пространстве.

```
cluster_assignments = (BEST_KMEANS.transform(shop_vectors)
    .select("row_id", F.col("prediction").alias("cluster")))

cluster_profiles = (shop_raw.join(cluster_assignments, "row_id")
    .groupBy("cluster")
    .agg(
        F.count("*").alias("n"),
        F.avg("Administrative").alias("administrative"),
```

```
        F.avg("Informational").alias("informational"),
        F.avg("ProductRelated").alias("product_pages"),
        F.avg("ProductRelated_Duration").alias("product_duration"),
        F.avg("BounceRates").alias("bounce_rate"),
        F.avg("ExitRates").alias("exit_rate"),
        F.avg(F.col("Revenue").cast("double")).alias("purchase_share")
    )
    .orderBy("cluster"))
cluster_profiles.show(truncate=False)

pca_model = PCA(k=2, inputCol="features",
outputCol="pca_features").fit(shop_vectors)
pca_rows = (pca_model.transform(BEST_KMEANS.transform(shop_vectors))
    .orderBy(sample_order, "row_id")
    .limit(4000)
    .select("prediction", "pca_features")
    .toPandas())
coords = np.vstack([v.toArray() for v in pca_rows["pca_features"]])

fig, ax = plt.subplots(figsize=(7, 5))
for cluster in sorted(pca_rows["prediction"].unique()):
    mask = pca_rows["prediction"].to_numpy() == cluster
    ax.scatter(coords[mask, 0], coords[mask, 1], s=9, alpha=0.35,
               label=f"Кластер {cluster}")
ax.set(xlabel="Первая главная компонента",
       ylabel="Вторая главная компонента",
       title="Проекция кластеров")
ax.grid(alpha=0.2)
ax.legend()
plt.tight_layout()
plt.savefig(OUT / "clusters_pca.png", dpi=160)
plt.show()
```

Таблица 11. Контрольные профили трёх кластеров

Кластер	n	Товарных страниц	Время на товарах	Bounce	Exit	Доля покупок
0	9046	26.3	962	0.010	0.033	14.9%
1	977	2.2	30	0.172	0.185	0.4%
2	2307	65.7	2600	0.007	0.022	23.9%

Таблица профилей показывает размер каждой группы и средние значения исходных признаков. Номер кластера не несёт постоянного смысла, так как может меняться от случайной инициализации, и интерпретация формируется только после сравнения строк. Долю покупок нужно читать вместе с размером группы, поскольку высокий процент в маленьком кластере может быть статистически нестабильным.

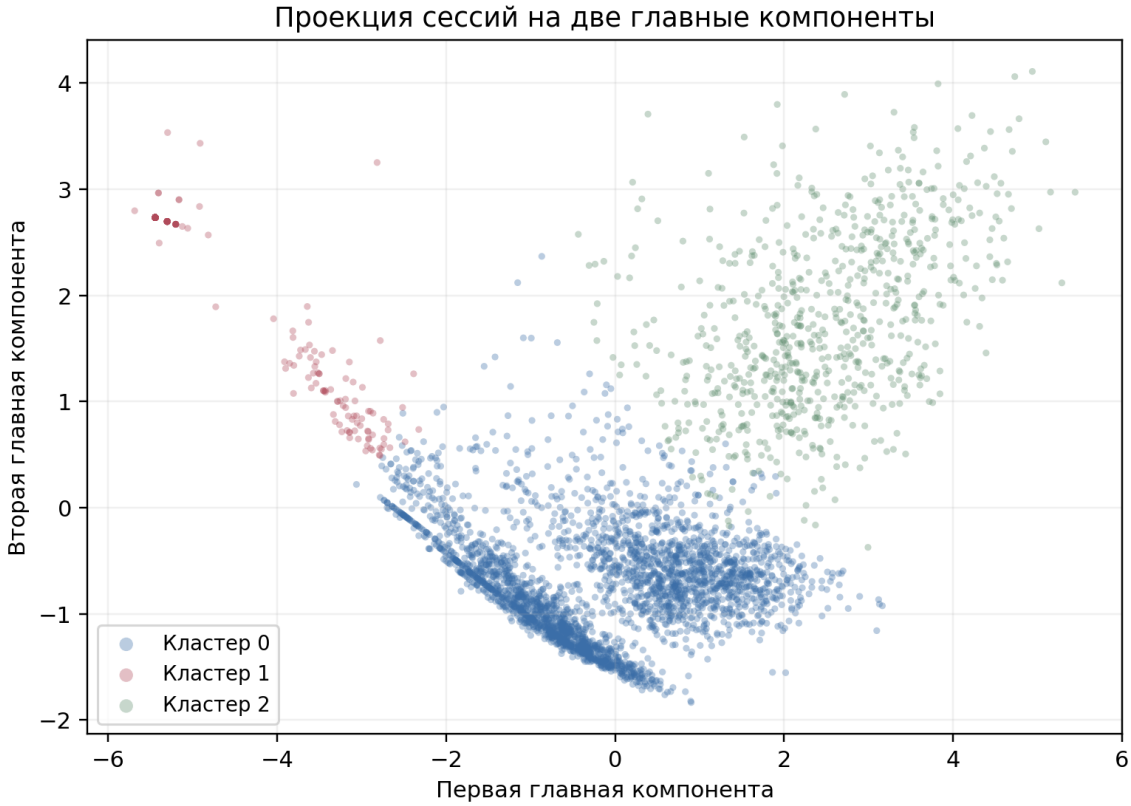


Рис. 13. Проекция сессий на две главные компоненты

На PCA-проекции видны три области различной плотности, но границы частично перекрываются. Метод KMeans назначает ближайший центр каждому объекту даже без визуально пустого промежутка между группами. Рисунок полезен для поиска общей геометрии и выбросов, но не заменяет метрику silhouette и профили.

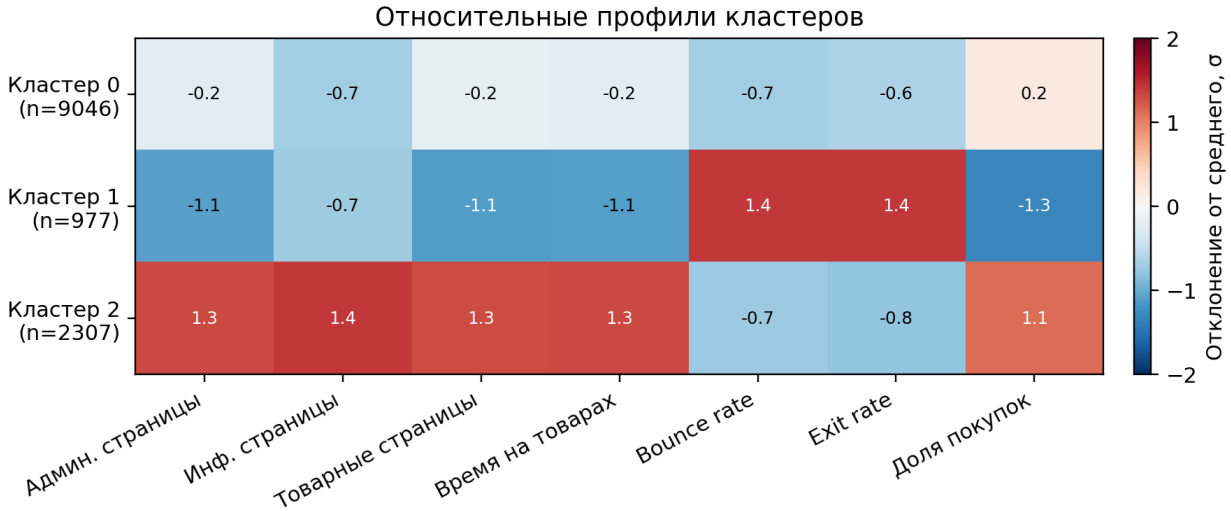


Рис. 14. Нормированные профили кластеров

Нормированный профиль показывает, какие признаки для каждого кластера выше или ниже общего среднего после масштабирования. Тёплые значения отражают повышенную активность, холодные — пониженную. Такая диаграмма удобна для

сопоставления формы групп, а таблица в исходных единицах нужна для содержательных названий и количественной интерпретации.
Кластер 1 содержит короткие сессии с высокими BounceRates и ExitRates, здесь покупкой заканчивается около 0,4% сессий. Кластер 2 отличается высокой активностью на товарных и административных страницах и имеет наибольшую долю покупок — около 23,9%. Кластер 0 является самой крупной промежуточной группой.

> Доля покупок не является внешней точностью кластеризации. Она используется только для содержательного описания после выбора k. Если подбирать k по Revenue, задача фактически превратится в скрытую классификацию.

7. Сохранение и повторное применение моделей

Сохранять только последний классификатор или регрессор недостаточно. Новые данные должны пройти через те же логарифмы, словари категорий, one-hot-кодирование, сборку вектора и масштабирование. Поэтому записывается весь PipelineModel. Рабочий порог классификации хранится отдельно: он относится к правилу применения score, а не к параметрам обученного случайного леса.
Блок сохраняет три полные цепочки: классификатор Online Shoppers, регрессор Seoul Bike и подготовку с KMeans. JSON-файл выступает паспортом применения: фиксирует seed, выбранные алгоритмы, порог, число кластеров и версию Spark. При повторном использовании PipelineModel загружается методом load, а значения из JSON не должны заменяться неявными переменными старого блокнота.

```
BEST_SHOP.write().overwrite().save((OUT / "shop_classifier").as_posix())
BEST_BIKE.write().overwrite().save((OUT / "bike_regressor").as_posix())

cluster_full_pipeline = PipelineModel(
    stages=cluster_prep_model.stages + [BEST_KMEANS]
)
cluster_full_pipeline.write().overwrite().save(
    (OUT / "shop_kmeans").as_posix()
)

config = {
    "seed": SEED,
    "classification_model": BEST_SHOP_NAME,
    "classification_threshold": SHOP_THRESHOLD,
    "regression_model": BEST_BIKE_NAME,
    "kmeans_k": BEST_K,
    "spark_version": spark.version
}
(OUT / "model_config.json").write_text(
    json.dumps(config, ensure_ascii=False, indent=2), encoding="utf-8"
```

```
)
print(json.dumps(config, ensure_ascii=False, indent=2))
```

```
{
  "seed": 42,
  "classification_model": "Случайный лес",
  "classification_threshold": 0.55,
  "regression_model": "Случайный лес",
  "kmeans_k": 3,
  "spark_version": "4.1.2"
}
```

Сохранённая модель воспроизводит преобразования, но не гарантирует неизменность качества при сдвиге данных. Для классификации необходимо контролировать долю покупок и пригодность порога, для регрессии — временной перенос и диапазон погоды, для KMeans — изменение центров и профилей. Номера кластеров при новом обучении могут переставиться и не являются постоянными бизнес-идентификаторами.

Практический блок 7.1. Освобождение кэша

```
for pred in shop_valid_predictions.values():
    pred.unpersist()
for df in [shop_split, shop_test_scores, bike_train, bike_valid,
           bike_test, bike_test_prediction, shop_vectors, cluster_sample]:
    df.unpersist()

print("Готово. Результаты сохранены в", OUT)
```

Кэш освобождается после завершения опытов. Останавливать SparkSession нужно только после сохранения снимков Spark UI и выполнения самостоятельной части.

8. Самостоятельная работа

В качестве самостоятельной работы предлагается решить задачу регрессии, классификации или кластеризации на основе Spark MLlib с использованием нескольких моделей машинного обучения (не менее двух моделей и сравнение с базовым решением). В качестве ответа необходимо подготовить Jupyter блокнот с процессом выполнения работы:

1. Загрузка данных в среду выполнения и определение Spark-схемы
2. Просмотр распределения ключей объектов для определения схемы разбиения выборки
3. Разбиение набора данных на тренировочный, валидационный и тестовый наборы, опираясь на смысл выборки (независимые объекты / временные данные).

4. Просмотр распределений предикторов и целевой переменной
5. Выбор предикторов и целевой переменной прогнозирования, определение точек возможной утечки данных.
6. Построение и оценка параметров моделей машинного обучения, подбор оптимальных гиперпараметров либо с помощью кросс-валидации, либо минимизируя функцию стоимости на валидационной выборке.
7. Оценка качества модели на тестовых данных с использованием метрики качества модели и визуализация работы модели на тестовой выборке.

К сдаче предоставляются выполненный Jupyter Notebook, отчёт и каталог результатов. Блокнот должен выполняться сверху вниз после перезапуска ядра без ручной подмены промежуточных переменных.

Таблица 12. Рекомендуемое распределение баллов

Критерий	Баллы
Корректное чтение и проверка данных	1
Разбиение без утечки и воспроизводимость	2
Классификация и интерпретация порога/ Регрессия, базовые правила и временная проверка/ Кластеризация, silhouette и профили	3
Графики и содержательные выводы	2
Итого	8

9. Контрольные вопросы

1. Чем параметр модели отличается от гиперпараметра? Приведите примеры для LR, RF и KMeans.
2. Почему train, valid и test нельзя заменить одним случайным разбиением train/test во всех задачах?
3. Что называется утечкой данных и почему она может дать очень высокую, но бесполезную метрику?
4. Почему числовые коды Browser, Region и TrafficType рассматриваются как категории?
5. Что обучают StringIndexer, StandardScaler и сам классификатор внутри Pipeline?
6. Почему accuracy около 0,85 почти ничего не доказывает для Online Shoppers?
7. Как меняются precision и recall при повышении порога решения?
8. Почему PR-AUC особенно полезна при редком положительном классе?
9. Что показывают коэффициенты логистической регрессии и почему их нельзя считать причинными эффектами?

10. Почему PageValues проверяется отдельным опытом? Как момент выдачи прогноза меняет список допустимых признаков?
11. Почему часы Functioning Day = No исключены из основной регрессии?
12. Зачем час кодируется одновременно через sin и cos?
13. В каком смысле PolynomialExpansion делает зависимость нелинейной?
14. Чем MAE отличается от RMSE? Какая метрика сильнее реагирует на пики ошибки?
15. Как понимать отрицательный R^2 на test?
16. Почему простое среднее по часу является более сильным baseline, чем глобальное среднее?
17. Почему day_index может помочь на одном году и стать проблемой при переносе на другой?
18. Почему trainingCost KMeans нельзя использовать как единственный критерий выбора k?
19. Что означает отрицательный, нулевой и положительный silhouette?
20. Почему Revenue нельзя включать в features кластеризации, если после обучения мы хотим сравнить долю покупок?
21. Почему PCA-проекция не доказывает, что кластеры полностью разделены в исходном восьмимерном пространстве?
22. Какие данные допустимо передавать через toPandas(), а какие должны оставаться распределёнными?
23. Что необходимо сохранить вместе с PipelineModel, чтобы решение можно было применить повторно?
24. Какие отличия численных метрик между двумя версиями Spark допустимы, а какие указывают на ошибку в данных или коде?

10. Список использованных источников

1. Online Shoppers Purchasing Intention Dataset. UCI Machine Learning Repository. DOI 10.24432/C5F88Q.
https://archive.ics.uci.edu/dataset/468/online+shoppers+purchasing+intention+dataset
2. Seoul Bike Sharing Demand. UCI Machine Learning Repository. DOI 10.24432/C5F62R.
https://archive.ics.uci.edu/dataset/560/seoul+bike+sharing+demand
3. Apache Spark. MLlib: DataFrame-based API, Pipeline, Estimator, Transformer and Model. https://spark.apache.org/docs/latest/api/python/reference/pyspark.ml.html
4. Apache Spark. Classification and regression with spark.ml.
https://spark.apache.org/docs/latest/ml-classification-regression.html

5. PySpark API: PolynomialExpansion. https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.feature.PolynomialExpansion.html
6. PySpark API: KMeans. https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.clustering.KMeans.html
7. scikit-learn: precision_recall_curve and silhouette_score. https://scikit-learn.org/stable/api/sklearn.metrics.html